Learn Cilium

eBPF e Cilium sem mistério: fundamentos e laboratório guiado para iniciantes

Teresina

2026

Learn Cilium

eBPF e Cilium sem mistério: fundamentos e laboratório guiado para iniciantes

Edição 0.2 de trabalho. Público-alvo: iniciantes em Linux, redes e Kubernetes.

Teresina
2026

APRESENTAÇÃO

Este livro de trabalho parte de três perguntas simples: em que lugar um comando roda, por onde uma conexão passa e como descobrir onde ela parou? Os capítulos C01–C03 apresentam Linux, rede e objetos Kubernetes antes do laboratório integrador. Os exercícios E01–E06 percorrem reconhecimento do ambiente, caminho saudável, observação com Hubble, política de ingresso, falha de selector e auditoria com limpeza.

Estado editorial: edição de trabalho. Os comandos e manifests desta edição não foram reproduzidos em um cluster. Saídas são marcadas como **ILUSTRATIVAS**; campos de observação real ficam vazios até uma execução admitida. O material não afirma imagem instalada, tráfego observado, aprendizagem de leitor ou validação operacional. Cilium stable 1.20.4 aparece como referência consultada em 28 de outubro de 2026, não como versão instalada. (Cilium Authors, 2026)

A prática só ocorre em ambiente Linux fornecido pelo instrutor, com reserva exclusiva e gates de admissão confirmados. O instrutor já preparou Cilium como CNI, Hubble Relay, políticas globais avaliadas, baseline protegido e capacidade adequada. Não instale nem altere Cilium, Hubble ou o cluster por este roteiro. Se um gate estiver desconhecido, pare antes de criar recursos.

Os manifests acompanhantes ficam em poc-k8s-fabric/k8s/p010-iniciantes/. Use-os junto com esta mesma revisão. Obtenha o repositório ou um pacote do instrutor, confira a revisão identificada por ele e não misture arquivos de versões diferentes. Os manifests deixam o campo namespace ausente de propósito; os comandos usam o namespace próprio com -n. Os três Deployments usam uma réplica cada. Imagens são referenciadas por digest de índice conferido em recibo; isso não prova que o ambiente as consiga baixar ou executar.

Este livro diferencia quatro lugares: o computador que você usa, a VM Linux, o cliente kubectl e o processo dentro do Pod. No roteiro, kubectl e Hubble são chamados na VM Linux; o pedido curl é executado dentro do Pod frontend; o API server e os agentes Cilium ficam no cluster. Um comando kubectl exec começa no cliente e executa o processo depois de – dentro do contêiner escolhido.

LISTA DE FIGURAS

Figura 1 – Linux separa processo, kernel, contêiner e lugar de execução 8
Figura 2 – Um nome, um endereço e uma conexão carregam informações diferentes . . 12
Figura 3 – Deployment, Pod, Service e EndpointSlice respondem perguntas diferentes . 16

Figura 4 – E01 verifica identidade e gates antes de abrir uma namespace nova 18
Figura 5 – E02 compara o caminho pelo proxy com o acesso direto 28
Figura 6 – E03 limita a observação ao namespace e à janela do exercício 32
Figura 7 – E04 permite dois trechos e bloqueia o atalho direto 38
Figura 8 – E05 separa a saúde do Pod da seleção pelo Service 41
Figura 9 – E06 mantém o recibo local separado do audit log do control plane 45
Figura 10 – O agente configura; o programa atua no hook 52
Figura 11 – Sequência temporal: loader, kernel e hook 53
Figura 12 – Decisão do verifier e limite da conclusão 54

LISTA DE QUADROS

Quadro 1 – Vocabulário em uma frase 52

LISTA DE TABELAS

Tabela 1 – Verificações e limites da conclusão 54

SUMÁRIO

1 LINUX SEM MEDO 7
1.1 Por que começar pelo Linux 7
1.2 Palavras antes do jargão 7
1.3 Terminal, shell e comandos 8
1.4 Exemplo e perguntas 9

2 UMA CONVERSA PELA REDE 11
2.1 Seguir a ida e a volta 11
2.2 Termos sem atalhos 11
2.3 Um endereço lógico não é o caminho inteiro 12

3 KUBERNETES EM TAMANHO PEQUENO 15
3.1 Objetivos e objetos que se encontram 15
3.2 Criação de Pod e escolha de endpoint 15
3.3 Preparar a prática e os terminais 17
3.4 E01 — conferir o ambiente antes de criar recursos 17
3.5 E02 — seguir uma requisição saudável 27
3.6 E03 — observar uma janela curta com Hubble 32
3.7 E04 — restringir ingress sem quebrar a cadeia 37
3.8 E05 — encontrar e restaurar um selector errado 41
3.9 E06 — auditar evidências e limpar somente o próprio namespace 45

Prévia: como eBPF responde a eventos 51
Um programa eBPF encontra seu evento 51
O que você vai aprender 51
Palavras antes do jargão 52
Um ponto de ligação, dois papéis 52
Da solicitação ao evento 53
Aceito ou rejeitado: o alcance do verifier 53
Pratique e confira 54

Referências 56

1 LINUX SEM MEDO

1.1 Por que começar pelo Linux

O Linux é o sistema operacional no qual o Cilium integra boa parte de sua rede. Entender onde termina um processo e começa o kernel evita tratar cada contêiner como uma máquina independente. Neste capítulo, você vai:

- distinguir um processo, o kernel e o sistema de arquivos usado por um contêiner;
- explicar para que servem namespaces e cgroups sem supor que um contêiner tem um kernel próprio;
- localizar comandos no computador, na VM Linux e em um Pod e interpretar seu resultado.

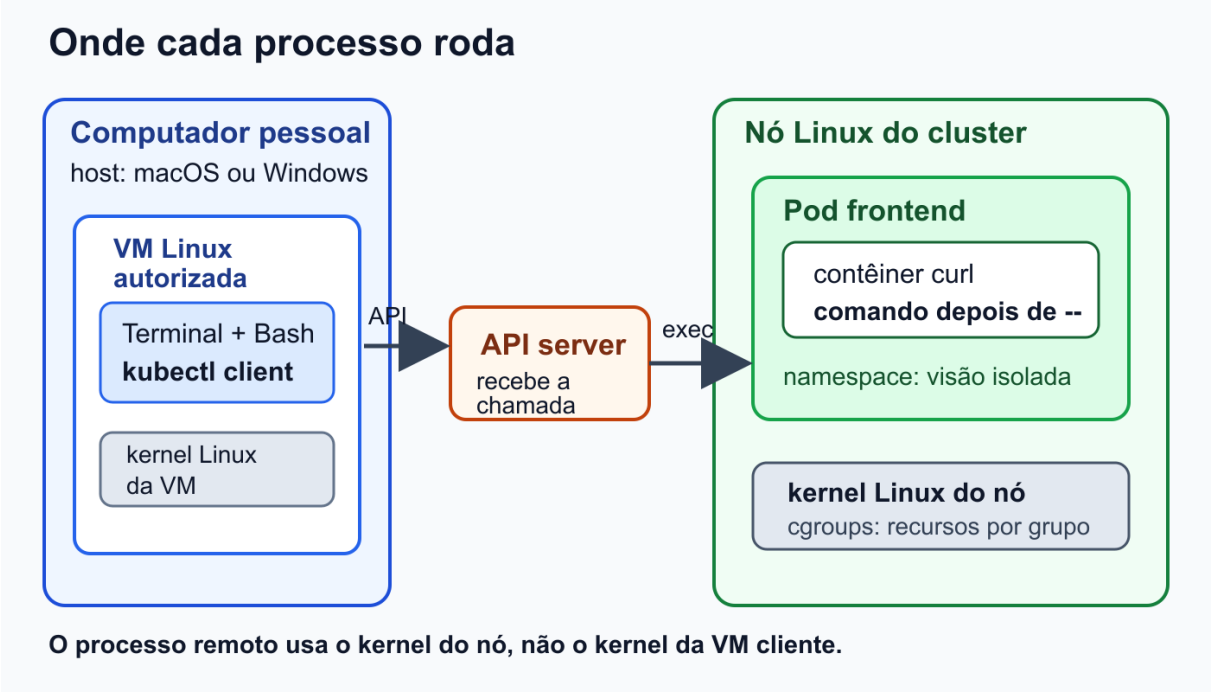
Pré-requisitos: nenhum. Não é preciso abrir terminal ou ter um cluster para ler os exemplos.

1.2 Palavras antes do jargão

O **kernel** é o núcleo que administra CPU, memória, dispositivos e chamadas de sistema. Um **processo** é um programa em execução, com um identificador e recursos associados. Um **contêiner** organiza um ou mais processos com uma visão isolada de recursos do Linux; ele usa o kernel do sistema que o hospeda. **Namespaces** alteram a visão de recursos como processos, rede e montagem de arquivos. **Cgroups** organizam e medem recursos de grupos de processos. Esses recursos ajudam a formar contêineres, mas não transformam cada contêiner em uma máquina virtual completa. (Linux man-pages project, 2026)

Uma **máquina virtual** inclui um sistema operacional convidado e seu próprio kernel virtualizado. Um contêiner Linux comum compartilha o kernel Linux do host. Uma analogia limitada: apartamentos compartilham partes de um prédio, enquanto cada morador tem um espaço próprio. A analogia para nos limites das diferenças entre um prédio físico e as regras técnicas de namespaces, cgroups e sistema de arquivos; não descreve o mecanismo de segurança completo.

Figura 1 – Linux separa processo, kernel, contêiner e lugar de execução



Fonte: elaboração própria (2026), com base em (Linux man-pages project, 2026), (Kubernetes Authors, 2026) e no roteiro E01.
Nota: é um modelo didático, não um diagrama de um cluster medido.

1.3 Terminal, shell e comandos

Um **terminal** é a interface onde você digita e lê texto. Um **shell**, como Bash, interpreta essa linha e chama programas. Uma **variável** guarda um valor durante a sessão; escrever LAB_NS=valor atribui um valor ao nome LAB_NS, e $LAB_NS pede o valor guardado. Aspas duplas mantêm junto um valor que tenha espaços e deixam variáveis serem expandidas. Aspas simples preservam o conteúdo literalmente. A barra / separa diretórios e também aparece em URLs e caminhos; seu significado depende do contexto. O escape com barra invertida muda como o shell interpreta o próximo caractere. (GNU Project, 2025)

Um **código de saída** é o número que o comando devolve ao shell: zero costuma indicar conclusão sem erro para aquele comando; valor diferente de zero indica falha segundo o próprio programa. Isso não é o mesmo que um código HTTP. Com curl –fail-with-body, uma resposta HTTP de erro também faz curl devolver código diferente de zero; sem a opção, curl pode terminar com código zero mesmo ao receber HTTP 502. Registre os dois sinais quando ambos existirem. (GNU Project, 2025)

A tecla **Ctrl-C** envia uma interrupção ao comando em primeiro plano daquele terminal. No exercício E03, a porta encaminhada roda em outro terminal; Ctrl-C nele encerra somente aquele encaminhamento. Não pressione Ctrl-C em um terminal que esteja aplicando ou apagando recursos.

1.4 Exemplo e perguntas

Leia os locais:

- HOST: computador pessoal. O kernel pode ser macOS; comandos Linux não ganham um kernel Linux por serem executados aqui.
- VM Linux: convidado Linux que tem o cliente kubectl e acessa o API server com a identidade fornecida.
- POD: unidade Kubernetes que reúne contêineres e compartilha uma identidade de rede local entre eles.
- processo no frontend: o curl executa no contêiner, mas kubectl exec é enviado pela VM ao API server.

Perguntas para recordar

1. Um contêiner Linux padrão tem um kernel próprio?
2. O que namespaces e cgroups organizam?
3. Onde roda o comando depois de kubectl exec … –?
4. Se curl devolveu código 22, isso informa diretamente o código HTTP exato?

Gabarito comentado: (1) não; um contêiner Linux comum compartilha o kernel Linux do host. (2) namespaces organizam visões de recursos; cgroups organizam e acompanham recursos de grupos de processos. (3) no contêiner e no Pod selecionados, não no shell da VM. (4) não; o código 22 sinaliza uma classe de resposta HTTP não bem-sucedida em curl, enquanto o status HTTP deve ser lido nos cabeçalhos. Um código de saída descreve o resultado reportado pelo processo, não substitui a evidência da aplicação.

Atividade guiada — 5 minutos, sem cluster: classifique cada ação como HOST, VM ou POD e diga se o número observado é código de saída ou status HTTP: (A) abrir um terminal no computador pessoal; (B) executar `kubectl get pods`; (C) interpretar o padrão esquemático abaixo; (D) ler o `HTTP/1.1 200` impresso por curl. Anote também qual linha atravessa o API server.

```
kubectl exec ... -- curl http://api/
```

Os pontos representam o nome do Pod; não copie a linha como um comando pronto.

Resultado esperado — ILUSTRATIVO: A ocorre no HOST; B ocorre na VM e conversa com o API server; C é iniciado na VM, mas o processo curl roda no POD; D é um status HTTP, não o código de saída. Nenhum cluster é necessário para completar a classificação.

Prática integrada C01→E01 — estimativa preliminar de 15–25 minutos, não medida: E01 aplica o mapa HOST/VM/POD, variáveis e aspas, código de saída e a regra de parar diante de uma identidade divergente. O tempo real depende da preparação do instrutor e das respostas do ambiente.

Erro comum: executar um comando no lugar errado e interpretar a falha como problema de rede. Antes de qualquer comando, diga em voz alta: "host, VM ou Pod?"

Resumo: o kernel executa processos; contêineres usam mecanismos de isolamento sobre um kernel; o shell interpreta comandos; local de execução e código de saída fazem parte do diagnóstico.

Glossário: kernel — núcleo do sistema operacional; processo — programa em execução; contêiner — ambiente organizado para processos; namespace — visão isolada de certos recursos; cgroup — agrupamento/controle de recursos; terminal — interface de texto; shell — intérprete de comandos; variável — nome que guarda valor; código de saída — resultado do comando para o shell.

Leituras: namespaces e cgroups no Linux; segurança de contêineres e contextos de segurança no Kubernetes; Bash: expansão, aspas e status de saída.

2 UMA CONVERSA PELA REDE

2.1 Seguir a ida e a volta

Uma aplicação conversa com outra usando nomes, endereços, portas e protocolos. Neste capítulo, você vai:

- diferenciar nome DNS, endereço IP, bloco CIDR e porta;
- narrar em ordem a resolução de nome, a abertura TCP e as formas de encerrar ou interromper uma conexão;
- explicar que uma resposta HTTP retorna pela conexão e que o caminho concreto depende do ambiente.

Pré-requisitos: reconhecer o shell, o processo e a VM Linux do C01. Não é necessário calcular sub-redes à mão.

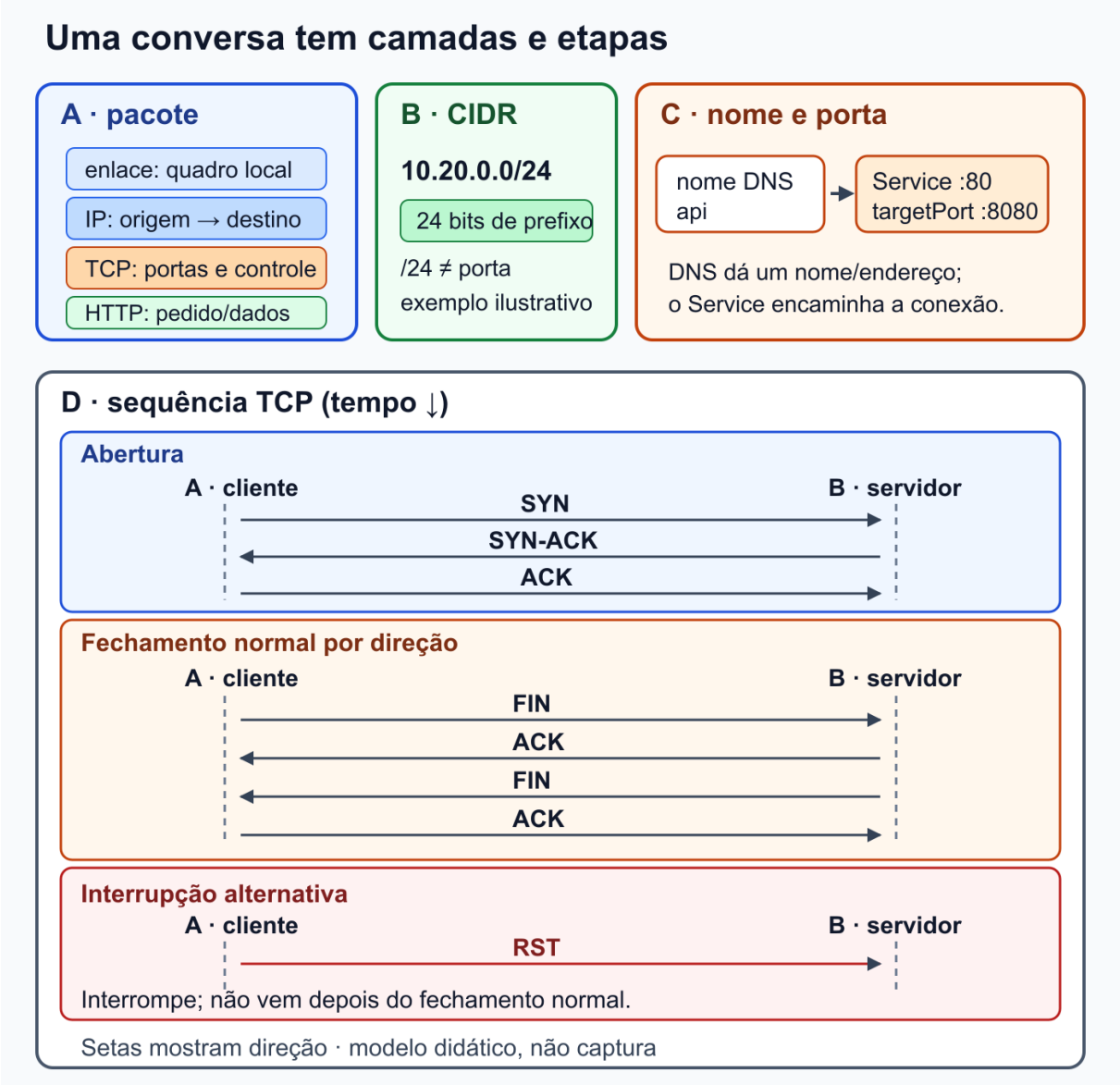
2.2 Termos sem atalhos

Um **nome DNS** é um nome consultado para encontrar dados associados, como o endereço de um Service Kubernetes. **DNS** responde a uma consulta de nome; não carrega por si só o conteúdo HTTP. Um **endereço IP** identifica um destino na rede durante determinado escopo e tempo. **CIDR** escreve um endereço seguido do tamanho do prefixo de rede, por exemplo 10.20.0.0/24; /24 conta bits do prefixo, não indica uma porta. Um **pacote** transporta dados e cabeçalhos de protocolos em camadas.

Uma **porta TCP** identifica um ponto de comunicação dentro de um host de rede; não é um endereço IP. **TCP** estabelece uma conexão confiável entre extremos. **HTTP** descreve pedidos e respostas de aplicação carregados sobre uma conexão; método, caminho e status pertencem a HTTP, não ao endereço DNS. (IETF, 1987) (IETF, 2006) (IETF, 2022)

SYN participa da abertura TCP; **ACK** confirma dados ou controles recebidos; **FIN** informa que aquele lado não enviará mais dados nessa conexão; **RST** sinaliza uma interrupção. Depois de enviar FIN, o lado ainda pode receber dados enquanto o outro lado não encerra o próprio envio. RST não é uma etapa do fechamento normal, e seu efeito depende do estado TCP. (IETF, 2022)

Figura 2 – Um nome, um endereço e uma conexão carregam informações diferentes



Fonte: elaboração própria (2026), com base em (IETF, 1987), (IETF, 2006), (IETF, 2022) e (Kubernetes Authors, 2026).
Nota: endereços e prefixos são ilustrativos; IPs de Pod não são universais.

2.3 Um endereço lógico não é o caminho inteiro

No laboratório, o cliente pergunta por api. O DNS do cluster pode resolver esse nome para o Service api no namespace atual. O Service representa um destino lógico estável para o cliente; a rede do cluster encaminha a conexão para um Pod elegível. O endereço IP concreto do Pod e o mecanismo que o alcança dependem do cluster e do CNI. Não calcule nem fixe IP de Pod no roteiro. (Kubernetes Authors, 2026)

O Service da API recebe na porta 80 e encaminha ao contêiner na porta 8080. A API NGINX aceita a conexão e abre outra conexão ao Service backend na porta 80; esse Service encaminha ao processo httpd do backend na porta 8080. A resposta volta pela conexão de cada

trecho. “Porta 80 para 8080” descreve o mapeamento do Service deste exemplo; não declara uma regra universal do Cilium. (Kubernetes Authors, 2026)

TCP costuma começar com SYN, SYN-ACK e ACK. Esse handshake abre a conexão; não é uma confirmação de que HTTP funcionará. A aplicação ainda pode responder 404, 502 ou outro status. Uma conexão pode ser bloqueada antes de haver resposta HTTP. Em sentido contrário, uma resposta HTTP pode chegar e ainda indicar erro de aplicação. Em HTTP/1.1, conexões podem persistir; receber uma resposta HTTP não significa, por si só, que TCP foi fechado. (IETF, 2022)

No fechamento normal ilustrado, A envia FIN para encerrar seu envio; B confirma com ACK e ainda pode enviar dados. Quando B terminar, envia FIN no sentido contrário, e A confirma com ACK. Assim, cada FIN encerra o envio em uma direção. Algumas mensagens de controle podem ser combinadas; a figura separa os passos para facilitar a leitura. Um RST pode interromper a conexão em vez desse fechamento ordenado; não é o passo seguinte ao par de FINs, e a validade da interrupção depende do estado TCP. Este é um modelo didático, não uma sequência universal. (IETF, 2022)

Uma analogia limitada: DNS se parece com consultar um catálogo para obter um endereço; TCP se parece com abrir uma conversa de ida e volta; HTTP é o formato combinado para pedir e responder. A analogia ajuda a separar etapas, mas não descreve encaminhamento de pacotes, balanceamento de Service ou regras de policy.

Atividade guiada — 7 minutos, sem cluster: use o cenário fixo do laboratório e trace `frontend → api:80 → Pod api:8080 → backend:80 → Pod backend:8080`. Marque onde ocorre DNS, quais Services recebem na porta 80, quais contêineres recebem na 8080 e onde NGINX abre uma segunda conexão. Depois desenhe uma seta de resposta para cada conexão. Em duas linhas do tempo, ordene a abertura `SYN A→B`, `SYN-ACK B→A`, `ACK A→B` e o fechamento `FIN A→B`, `ACK B→A`, `FIN B→A`, `ACK A→B`. Marque qual lado ainda pode receber dados depois do primeiro FIN. Em um cartão separado, desenhe `RST A→B` e anote que ele interrompe, sem colocá-lo depois do fechamento normal.

Resultado esperado — ILUSTRATIVO: os nomes `api` e `backend` identificam Services; cada Service recebe na porta 80 e encaminha para a porta 8080 do Pod selecionado. O frontend abre a primeira conexão com a API; NGINX abre a segunda com backend. Cada resposta percorre sua conexão de volta. Isso é o modelo dos manifests, não uma medição do cluster.

Prática integrada C02→E02 — estimativa preliminar de 15–25 minutos, não medida: E02 aplica DNS, Service, portas, duas conexões TCP e resposta HTTP ao caminho dos manifests. A estimativa exclui espera extraordinária por imagem ou assistência e não é um tempo observado.

Perguntas para recordar

1. O que /24 mede em 10.20.0.0/24?

2. DNS transporta o corpo da resposta HTTP?
3. Qual é a diferença entre porta do Service e targetPort neste laboratório?
4. SYN, SYN-ACK e ACK abrem ou fecham a conexão?
5. No fechamento normal, o que cada FIN encerra? Onde entra um RST?

Gabarito comentado: (1) 24 bits do prefixo da rede. (2) não; DNS responde consultas de nome e a conversa HTTP usa uma conexão posterior. (3) o cliente chama o Service na 80; o Service direciona ao contêiner na 8080. (4) abrem. (5) FIN encerra o envio de quem o envia; o outro lado confirma e ainda pode enviar até mandar seu próprio FIN. RST é uma interrupção alternativa, não uma etapa depois do fechamento normal, e depende do estado TCP. As portas e IPs devem ser lidos no contexto e não copiados como valores universais.

Erro comum: dizer "o DNS falhou" quando o nome resolveu, mas o Service não tem endpoint pronto. Separar consulta DNS, conexão TCP, rota até o destino e resposta HTTP produz hipóteses diferentes.

Resumo: DNS resolve nomes; IPs identificam destinos; CIDR descreve prefixos; TCP abre conexões, pode encerrá-las por direção com FIN ou interrompê-las com RST; HTTP troca pedidos e respostas sem determinar sozinho quando TCP fecha.

Glossário: DNS — sistema de nomes; endereço IP — endereço de rede; CIDR — endereço com prefixo de rede; porta — identificador de comunicação TCP/UDP; SYN — controle usado na abertura TCP; ACK — confirmação; FIN — encerramento do envio em uma direção; RST — interrupção TCP; HTTP — protocolo de pedidos e respostas; status HTTP — resultado da resposta da aplicação.

Leituras: RFC 1034 (DNS); RFC 4632 (CIDR); RFC 9110 (HTTP Semantics); RFC 9293 (TCP); DNS e Services no Kubernetes.

3 KUBERNETES EM TAMANHO PEQUENO

3.1 Objetivos e objetos que se encontram

Kubernetes descreve o estado desejado por objetos. Um **cluster** contém o plano de controle e nós que executam Pods. Um **Pod** é a unidade programada no nó; contêineres do mesmo Pod compartilham rede local. Um **Deployment** pede réplicas de Pods e as mantém. Uma **label** é uma chave e valor que identifica um objeto; um **selector** escolhe objetos que têm as labels correspondentes. Label ajuda a agrupar, não autentica uma pessoa ou workload. (Kubernetes Authors, 2026)

Um **Service** fornece um nome e destino lógico para um conjunto selecionado de Pods. Uma **EndpointSlice** registra partes dos destinos de rede associados ao Service, incluindo endereços e condições como pronto ou encerrando. Um **CNI** integra a configuração de rede dos Pods; Cilium é o CNI já instalado neste roteiro. **NetworkPolicy** declara comunicações permitidas para Pods selecionados. **ConfigMap** entrega configuração não secreta. **Probe** é uma verificação periódica de uma aplicação; readiness influencia se o Pod recebe tráfego de Service, enquanto liveness pode levar à reinicialização. (Kubernetes Authors, 2026)

Ao final, você poderá:

- relacionar Node, Pod, Deployment, Service, selector e EndpointSlice;
- apontar quando a rede do Pod, a resolução DNS e o encaminhamento do Service têm papéis diferentes;
- explicar o que as probes verificam e o que uma evidência de Hubble não mostra.

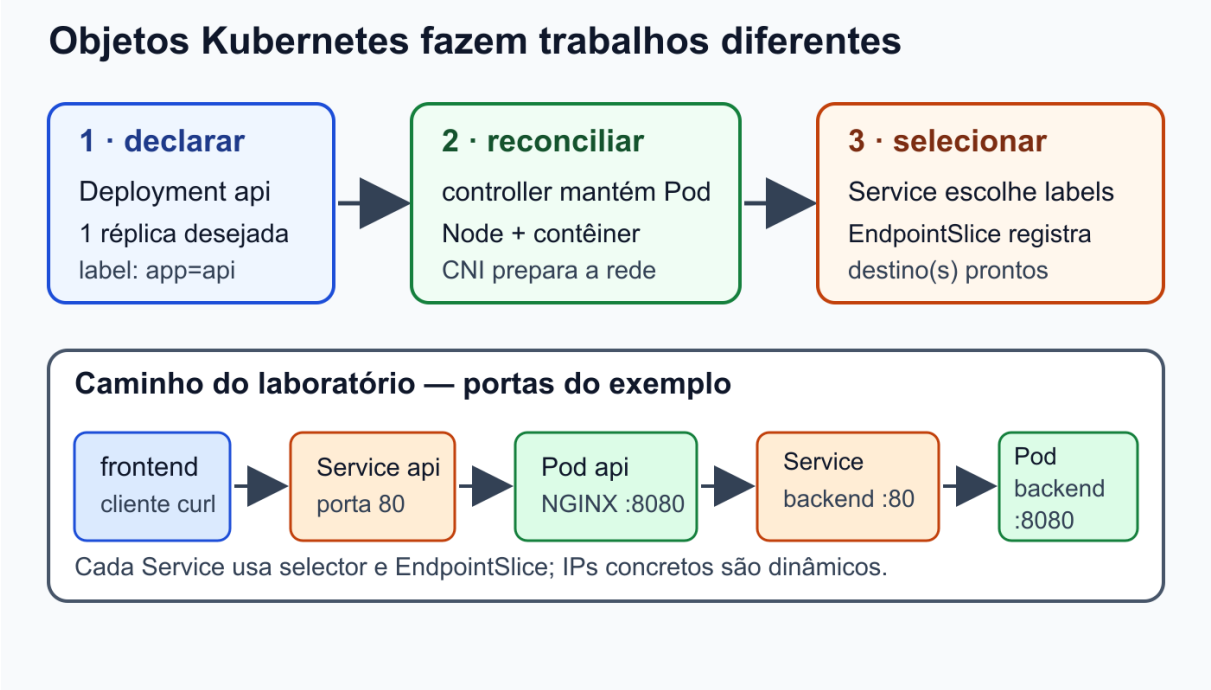
Pré-requisitos: vocabulário dos capítulos anteriores. O cenário usa apenas três Pods e dois Services internos; não instala componentes do cluster.

Analogia limitada: Deployment lembra uma receita de quantas cópias manter; Service lembra um endereço de atendimento. A receita não descreve cada pacote, e o endereço lógico não é uma pessoa nem contém a aplicação.

3.2 Criação de Pod e escolha de endpoint

O plano de controle recebe a intenção do Deployment. Um controlador mantém o conjunto de Pods; o scheduler escolhe um nó; o agente daquele nó e o CNI preparam a interface de rede do Pod. O Deployment não escolhe uma rota HTTP, o Service não inicia um contêiner e o CNI não substitui a aplicação. (Kubernetes Authors, 2026)

Figura 3 – Deployment, Pod, Service e EndpointSlice respondem perguntas diferentes



Fonte: elaboração própria (2026), com base em (Kubernetes Authors, 2026) e no percurso E02.
Nota: nomes são do laboratório, endereço de Pod e topologia permanecem dinâmicos.

O laboratório tem quatro papéis, mas três Deployments: frontend é um cliente curl ocioso; api é NGINX funcionando como proxy reverso; backend é BusyBox httpd servindo arquivos estáticos. “API” aqui nomeia um proxy didático, não uma API de negócio. O Service backend deve existir antes de o Deployment api iniciar: o NGINX usa o nome DNS backend em sua configuração. O Service api também recebe na porta 80 e aponta para o contêiner NGINX em 8080. (Kubernetes Authors, 2026) (NGINX Container Project, 2026)

A API tem uma rota local /ready que responde se o próprio processo NGINX está servindo. Ela não testa o backend. O backend fornece /health; a probe HTTP do Kubernetes consulta esse arquivo estático por httpd. O frontend executa sleep infinity e não é servidor; não recebe uma falsa probe HTTP que diga se a cadeia inteira está saudável. As sondagens HTTP devem ser interpretadas como uma verificação do endpoint especificado, não como teste da aplicação toda. (Kubernetes Authors, 2026)

Cada Pod usa usuário não root, seccomp RuntimeDefault, capabilities descartadas, sem escalada de privilégio, sistema de arquivos raiz somente leitura quando viável, recursos declarados e sem montagem de token do ServiceAccount. O ServiceAccount do exercício não recebe Role ou RoleBinding. Os contêineres NGINX e BusyBox usam volumes de configuração apenas para leitura; NGINX recebe /tmp gravável para PID e arquivos temporários exigidos por sua imagem sem abrir o resto do sistema de arquivos. A imagem NGINX unprivileged escuta em 8080. (Kubernetes Authors, 2026) (NGINX Container Project, 2026)

Exemplo ilustrativo de encadeamento

1. frontend consulta api dentro do mesmo namespace.
2. DNS localiza o Service api.
3. O Service encaminha a conexão à porta 8080 do Pod api.
4. O processo NGINX encaminha de verdade para http://backend:80.
5. Service backend escolhe Pod backend e encaminha para 8080.
6. BusyBox httpd devolve o arquivo; NGINX devolve corpo e cabeçalho X-Lab-Route.

Cada passo pode falhar por motivos diferentes. Status Ready do backend não prova que o Service o seleciona; um Service existente não prova que há EndpointSlice pronto; e um Pod com processo vivo não prova que a resposta da cadeia foi correta.

3.3 Preparar a prática e os terminais

Os exercícios são para uma VM Linux e um cluster que o instrutor reservou para esta turma. O computador do aluno pode ser macOS ou Windows; os comandos kubectl, cilium e hubble desta seção rodam na VM. O cliente HTTP roda dentro do Pod frontend por kubectl exec. Não cole os comandos abaixo no computador pessoal. O instrutor fornece a revisão do guia, o diretório de manifests, a identidade de acesso, o namespace livre e os dados de conferência do cluster.

Um comando termina com código de saída. Leia a mensagem e o código antes de continuar. Para ver o código imediatamente após um comando, execute echo $? na mesma janela. Em curl, guarde também o status HTTP impresso nos cabeçalhos: código de saída e status HTTP respondem perguntas diferentes. Aspas duplas em variáveis como “$LAB_NS” mantêm o valor como uma unidade; aspas simples preservam texto literal. A barra invertida no fim de uma linha continua o comando na próxima linha. Ctrl-C interrompe o processo em primeiro plano do terminal ativo; no encaminhamento Hubble, use-o somente na janela que o abriu.

Os manifests não declaram namespace. Cada comando que altera ou consulta objetos namespaced usa -n “$LAB_NS”. Uma namespace nova deve ser criada para cada rodada, nunca adotada de outra turma. Se o terminal for fechado, não reconstrua UID, run-id ou contexto por suposição: pare e peça ao instrutor para recuperar o recibo.

3.4 E01 — conferir o ambiente antes de criar recursos

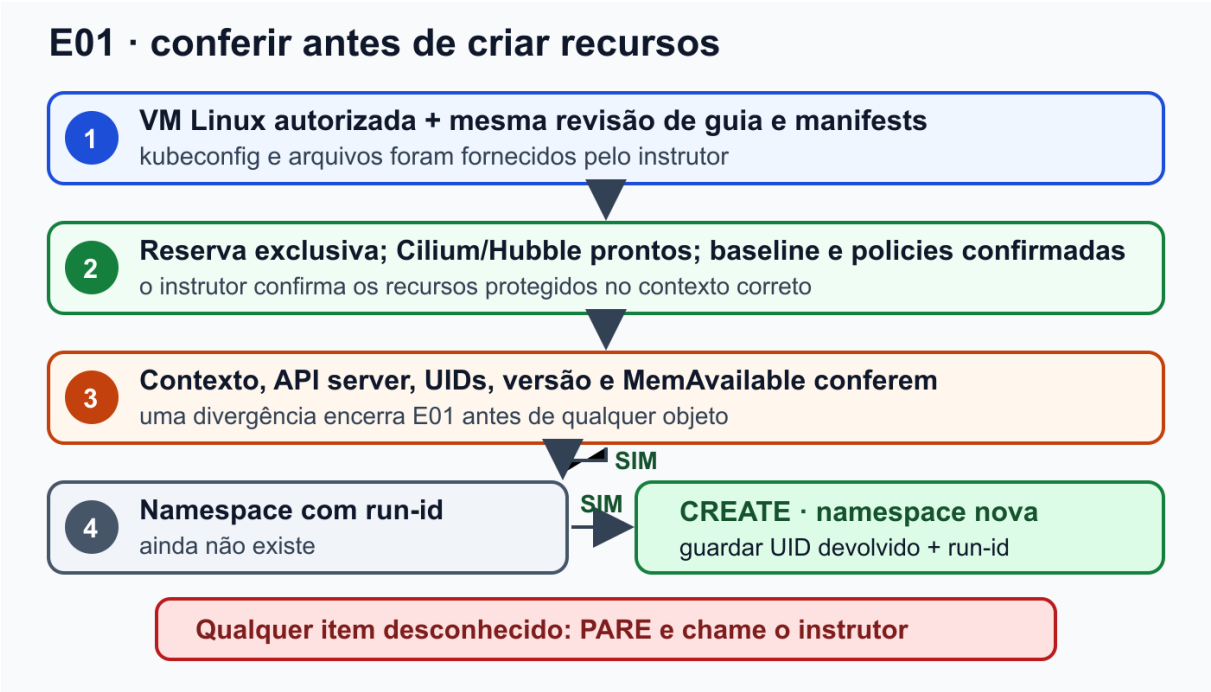
Objetivo: confirmar a identidade explícita do cluster e da namespace, os gates de capacidade e reserva, o estado inicial e a revisão dos arquivos antes de criar o namespace isolado.

Pré-requisitos: acesso à VM autorizado pelo instrutor; reserva exclusiva confirmada para o exercício; revisão exata do guia e manifests; Cilium já configurado; Hubble Relay já

disponível; modo de conexão e compatibilidade do cliente Hubble declarados pelo instrutor; policies globais e recursos protegidos avaliados pelo instrutor; pelo menos 4 GiB disponíveis em MemAvailable na VM. Qualquer item desconhecido significa parar antes da criação.

Previsão: as versões e identidades conferem com os valores fornecidos; o namespace escolhido ainda não existe; os objetos de baseline e o estado das policies são reconhecidos pelo instrutor; a cópia de manifests tem hashes registráveis. E01 não instala, ativa nem muda Cilium, Hubble, políticas globais ou componentes do cluster.

Figura 4 – E01 verifica identidade e gates antes de abrir uma namespace nova



Fonte: elaboração própria (2026), com base em (Kubernetes Authors, 2026) e no processo de admissão definido para este laboratório. Nota: os gates são verificações prévias; a figura não registra nem certifica um ambiente executado.

Passos explicados

1. Confirme com o instrutor a reserva exclusiva, os gates de Cilium/Hubble, as policies globais, a linha de base protegida e a capacidade. A reserva evita que outra turma altere os mesmos objetos durante E04 e E05. Não use comandos de uma turma anterior como atalho.

2. Na VM Linux, entre na cópia fornecida e confira que os manifests são da mesma revisão indicada pelo instrutor. O primeiro comando mostra o diretório atual; o segundo lista os YAML; o terceiro calcula uma soma por arquivo, sem ler credenciais:

```
pwd
find poc-k8s-fabric/k8s/p010-iniciantes -maxdepth 1 -type f -print
```

```
sha256sum poc-k8s-fabric/k8s/p010-iniciantes/*.yaml
```

3. Copie os valores de identidade fornecidos pelo instrutor para a sessão Bash. O caminho do kubeconfig deve ser absoluto. O arquivo contém credenciais e nunca deve ser impresso, anexado ao recibo ou copiado para os Pods. Escolha um nome novo para a rodada, usando somente letras minúsculas, dígitos e hífens no namespace; substitua os exemplos por valores reais:

```
export LAB_KUBECONFIG='/caminho/absoluto/fornecido/pelo/instrutor'
export LAB_CONTEXT='contexto-fornecido-pelo-instrutor'
export LAB_API_SERVER='https://api-fornecida-pelo-instrutor:6443'
export LAB_CLUSTER_UID='uid-kube-system-fornecido-pelo-instrutor'
export LAB_CILIUM_NS='namespace-cilium-fornecido-pelo-instrutor'
export LAB_NS='p010-aluno-20261028-a1b2'
export LAB_RUN_ID='20261028-a1b2'
export LAB_GUIDE_REVISION='revisao-fornecida-pelo-instrutor'
export LAB_DIR="$PWD/poc-k8s-fabric/k8s/p010-iniciantes"
export LAB_GUIDE="$PWD/poc-k8s-fabric/docs/ebpf-cilium-iniciantes.md"
LAB_DIR_REL='poc-k8s-fabric/k8s/p010-iniciantes'
LAB_GUIDE_REL='poc-k8s-fabric/docs/ebpf-cilium-iniciantes.md'
```

4. Defina o atalho k para sempre passar as duas opções de identidade, e record para salvar saída e código de retorno no recibo privado. Cada entrada identifica a revisão do guia, exercício, passo e rótulo. Não expande argumentos que possam revelar caminhos locais ou valores sensíveis. O teste abaixo para se faltar variável ou se o kubeconfig não for absoluto. Não execute os passos seguintes se as funções k e record não estiverem definidas neste terminal Bash:

```
for name in \
  LAB_KUBECONFIG LAB_CONTEXT LAB_API_SERVER LAB_CLUSTER_UID \
  LAB_CILIUM_NS LAB_NS LAB_RUN_ID LAB_GUIDE_REVISION LAB_DIR \
  LAB_GUIDE LAB_DIR_REL LAB_GUIDE_REL; do
  if [[ -z "${!name:-}" ]]; then
    printf 'Falta preencher %s; pare e peça ao instrutor.\n' \
      "$name" >&2
    exit 1
  fi
done
```

```
if [[ "$LAB_KUBECONFIG" != /* ]]; then
  printf 'LAB_KUBECONFIG deve ser um caminho absoluto.\n' >&2
  exit 1
fi
if [[ ! "$LAB_GUIDE_REVISION" =~ ^[A-Za-z0-9._/-]+$ ]]; then
  printf 'Revisão do guia deve ser tag ou commit sem espaços.\n' >&2
  exit 1
fi
if [[ ! "$LAB_NS" =~ ^p010-[a-z0-9]([-a-z0-9]{0,56}[a-z0-9])?$ ||
      ! "$LAB_RUN_ID" =~ ^[a-z0-9]([-a-z0-9]{0,61}[a-z0-9])?$ ]]; then
  printf 'LAB_NS ou LAB_RUN_ID não atende ao formato combinado.\n' \
    >&2
  exit 1
fi
if [[ ! -r "$LAB_KUBECONFIG" || ! -d "$LAB_DIR" ||
      ! -r "$LAB_GUIDE" || ! -d "$LAB_DIR_REL" ||
      ! -r "$LAB_GUIDE_REL" ]]; then
  printf 'Kubeconfig, guia ou diretório não está acessível.\n' \
    >&2
  exit 1
fi
k() {
  kubectl --kubeconfig "$LAB_KUBECONFIG" \
    --context "$LAB_CONTEXT" "$@"
}
umask 077
LAB_RECEIPT_DIR="$HOME/lab-receipts/$LAB_RUN_ID"
if [[ -e "$LAB_RECEIPT_DIR" ]]; then
  printf 'Recibo existente; escolha outro run-id com o instrutor.\n' \
    >&2
  exit 1
fi
mkdir -m 700 -p "$LAB_RECEIPT_DIR" || exit 1
LAB_RECEIPT="$LAB_RECEIPT_DIR/receipt.md"
cat > "$LAB_RECEIPT" <<EOF || exit 1
# Recibo local do laboratório

Revisão do guia: $LAB_GUIDE_REVISION
Início UTC: $(date -u +%Y-%m-%dT%H:%M:%SZ)
```

```
VM Linux: $(uname -srm)
Contexto esperado: $LAB_CONTEXT
API server esperado: $LAB_API_SERVER
UID kube-system esperado: $LAB_CLUSTER_UID
Namespace/run-id: $LAB_NS / $LAB_RUN_ID
Modo Hubble: preencher após declaração do instrutor

Não incluir kubeconfig, token, certificado, chave, Secret ou dado
pessoal. Resultados vazios são NOT_RUN, nunca suposições.

E01 — gates, versão, memória e criação
Esperado:
Observado:
Códigos de saída:
UID namespace criado:

E02 — cadeia e probes locais
Esperado:
Observado:
Códigos de saída:

E03 — janela Hubble e modo de conexão declarado
Esperado:
Observado:
Códigos de saída:

E04 — policy e contraprovas
Esperado:
Observado:
Códigos de saída:

E05 — selector e restauração
Esperado:
Observado:
Códigos de saída:

E06 — hashes, versões, imageID, arquitetura e cleanup
Esperado:
Observado:
```

```
Códigos de saída:
ImageIDs e arquitetura: preencher somente após execução admitida
Resultado do cleanup:
EOF
if [[ ! -f "$LAB_RECEIPT" ]]; then
  printf 'Recibo local ausente; pare antes de consultar o cluster.\n' \
    >&2
  exit 1
fi
record() {
  local label="$1"
  local command_rc
  local -a header_status
  local -a output_status
  local -a result_status
  shift
  RECORD_LOG_FAILED=0
  RECORD_COMMAND_RC=0
  if [[ -z "${LAB_GUIDE_REVISION:-}" ||
        -z "${RECORD_STEP:-}" ]]; then
    RECORD_LOG_FAILED=1
    printf 'Falta revisão ou passo para identificar o comando.\n' >&2
    return 125
  fi
  printf '\n## guia=%s | passo=%s | %s\n' \
    "$LAB_GUIDE_REVISION" "$RECORD_STEP" "$label" \
    | tee -a "$LAB_RECEIPT"
  header_status=("${PIPESTATUS[@]}")
  if (( header_status[0] != 0 || header_status[1] != 0 )); then
    RECORD_LOG_FAILED=1
    return 125
  fi
  "$@" 2>&1 | redact_receipt_output | tee -a "$LAB_RECEIPT"
  output_status=("${PIPESTATUS[@]}")
  command_rc=${output_status[0]}
  RECORD_COMMAND_RC="$command_rc"
  if (( output_status[1] != 0 || output_status[2] != 0 )); then
    RECORD_LOG_FAILED=1
    return 125
```

```
  fi
  printf 'exit_code=%s\n' "$command_rc" | tee -a "$LAB_RECEIPT"
  result_status=("${PIPESTATUS[@]}")
  if (( result_status[0] != 0 || result_status[1] != 0 )); then
    RECORD_LOG_FAILED=1
    return 125
  fi
  return "$command_rc"
}
redact_receipt_output() {
  local line
  local path
  while IFS= read -r line || [[ -n "$line" ]]; do
    for path in \
      "${LAB_KUBECONFIG:-}" \
      "${HUBBLE_TLS_CA_CERT_FILES:-}" \
      "${HUBBLE_TLS_CLIENT_CERT_FILE:-}" \
      "${HUBBLE_TLS_CLIENT_KEY_FILE:-}"; do
      if [[ -n "$path" ]]; then
        line=${line//"$path"/'[caminho local omitido]'}
      fi
    done
    printf '%s\n' "$line"
  done
}
```

5. Compare a versão do cliente e do servidor, o contexto selecionado por –context, o endpoint e o UID da namespace kube-system com o registro recebido. O UID de kube-system é uma impressão de identidade fornecida pelo instrutor, não um nome global padronizado para cluster. Salve a versão no recibo e encerre em qualquer erro:

```
RECORD_STEP='E01.5'
record 'kubectl version' k version || exit 1
actual_context="$(k config view --minify \
  -o jsonpath='{.contexts[0].name}')" || exit 1
actual_server="$(k config view --minify \
  -o jsonpath='{.clusters[0].cluster.server}')" || exit 1
actual_cluster_uid="$(k get namespace kube-system \
  -o jsonpath='{.metadata.uid}')" || exit 1
```

```
printf 'Contexto observado: %s\nAPI server observado: %s\n' \
  "$actual_context" "$actual_server" \
  | tee -a "$LAB_RECEIPT" || exit 1
printf 'UID kube-system observado: %s\n' \
  "$actual_cluster_uid" \
  | tee -a "$LAB_RECEIPT" || exit 1
if [[ "$actual_context" != "$LAB_CONTEXT" ||
      "$actual_server" != "$LAB_API_SERVER" ||
      "$actual_cluster_uid" != "$LAB_CLUSTER_UID" ]]; then
  printf 'Identidade divergente; pare e chame o instrutor.\n' \
    >&2
  printf 'Identidade divergente; criação não autorizada.\n' \
    >> "$LAB_RECEIPT"
  exit 1
fi
  printf 'Contexto, API server e UID conferidos.\n' \
  | tee -a "$LAB_RECEIPT" || exit 1
```

6. Verifique a capacidade da VM, usando o valor MemAvailable do Linux. O operador || exit 1 faz o shell realmente parar se awk não encontrar memória suficiente. Depois confira que o nome novo não aparece na lista. Uma falha de conexão também interrompe; não a interprete como namespace ausente:

```
RECORD_STEP='E01.6'
record 'MemAvailable' awk '
  /^MemAvailable:/ {
    print $2 " KiB disponíveis"
    if ($2 < 4194304) exit 1
    found = 1
  }
  END { if (!found) exit 1 }
' /proc/meminfo || exit 1
existing_namespaces="$(k get namespaces \
  -o jsonpath='{range .items[*]}{.metadata.name}{"\n"}{end}')" \
  || exit 1
if printf '%s\n' "$existing_namespaces" | grep -Fxq -- "$LAB_NS"; then
  printf 'Namespace já existe; não a adote. Escolha outro run-id.\n' \
    >&2
  printf 'Namespace pré-existente; criação não tentada.\n' \
```

```
    >> "$LAB_RECEIPT"
  exit 1
fi
printf 'Namespace ausente antes da criação.\n' \
  | tee -a "$LAB_RECEIPT" || exit 1
```

7. Peça ao instrutor a confirmação da reserva exclusiva, capacidade, Cilium/Hubble, baseline protegida e policies globais. A baseline pode viver em outro contexto: não consulte nem altere esse contexto com a identidade desta prática. Registre a confirmação, a revisão e os hashes dos arquivos no recibo local:

```
RECORD_STEP='E01.7'
printf 'Confirmação, revisão e UTC: preencher com o instrutor.\n' \
  | tee -a "$LAB_RECEIPT" || exit 1
record 'hashes do guia e manifests' \
  sha256sum "$LAB_GUIDE_REL" "$LAB_DIR_REL"/*.yaml || exit 1
```

8. Somente depois de todos os gates, crie a namespace com um documento YAML enviado pela entrada padrão. create -f cria apenas um objeto novo; o servidor devolve o UID na mesma resposta. Se já existir ou a criação falhar, o comando retorna erro e exit 1 interrompe o shell; não tente apply nem outro contexto:

```
create_label='kubectl create -f - -o jsonpath={.metadata.uid}'
printf '\n## guia=%s | passo=E01.8 | %s\n' \
  "$LAB_GUIDE_REVISION" "$create_label" \
  | tee -a "$LAB_RECEIPT"
create_header_status=("${PIPESTATUS[@]}")
if (( create_header_status[0] != 0 ||
      create_header_status[1] != 0 )); then
  printf 'Recibo não confirmou criação; namespace não criada.\n' \
    >&2
  exit 1
fi
if LAB_NS_UID="$(k create -f - \
  -o jsonpath='{.metadata.uid}' <<EOF
apiVersion: v1
kind: Namespace
metadata:
```

```
  name: ${LAB_NS}
  labels:
    lab.learn-cilium.org/run-id: "${LAB_RUN_ID}"
EOF
)"; then
  :
else
  create_rc=$?
  printf 'Namespace create falhou; exit_code=%s. Pare.\n' \
    "$create_rc" | tee -a "$LAB_RECEIPT" || exit 1
  exit "$create_rc"
fi
if [[ -z "$LAB_NS_UID" ]]; then
  printf 'A criação não devolveu UID; pare e chame o instrutor.\n' \
    >&2
  exit 1
fi
printf 'namespace=%s uid=%s run-id=%s exit_code=0\n' \
  "$LAB_NS" "$LAB_NS_UID" "$LAB_RUN_ID" \
  | tee -a "$LAB_RECEIPT" || exit 1
```

Os valores LAB_NS e LAB_RUN_ID foram limitados a caracteres seguros no passo 4 para que o shell os possa inserir no YAML. Registre a revisão, UTC e UID retornado. O recibo foi criado antes da primeira chamada ao API server e permanece local com permissão restrita.

Esperado — ILUSTRATIVO: versão de cliente/servidor e identidade conferem; capacidade atende o limite definido para a VM; namespace ainda não existe; o instrutor confirma baseline e policies; o namespace criado tem UID próprio e label de run-id. Nenhum desses resultados foi coletado neste documento.

Observação real — NOT_RUN: revisão do guia: _____; hora UTC: _____; VM/versão: _____; contexto/API server/UID kube-system: _____; MemAvailable: _____; gates e confirmação do instrutor: _____; namespace/run-id/UID: _____; hashes dos YAML: _____; código de saída de cada etapa: _____.

Interpretação: uma identidade consistente reduz o risco de atuar no cluster errado; a reserva e a avaliação de policies reduzem interferência; um namespace novo delimita os objetos namespaced. Esses sinais não provam ausência de toda política externa nem substituem a confirmação do instrutor.

Se falhar: qualquer divergência de contexto, API server, UID, memória, versão, namespace existente, gate desconhecido ou erro de conexão encerra E01 sem criar recursos. Mantenha

o recibo do erro e chame o instrutor. Não tente trocar contexto para um padrão, completar uma instalação ou reutilizar uma namespace.

Sucesso: todos os gates foram confirmados, identidade explícita conferiu, o namespace não existia antes e foi criado com run-id; o UID e os hashes ficaram registrados sem material secreto.

Gabarito comentado: a checagem de endpoint e UID responde "qual cluster"; o nome da namespace com run-id responde "qual rodada"; a reserva responde "quem mais pode estar mudando o ambiente"; e a linha de base/policies respondem "o que não deve ser quebrado". Uma namespace ajuda a limitar nomes, mas não isola todo recurso nem controla políticas globais.

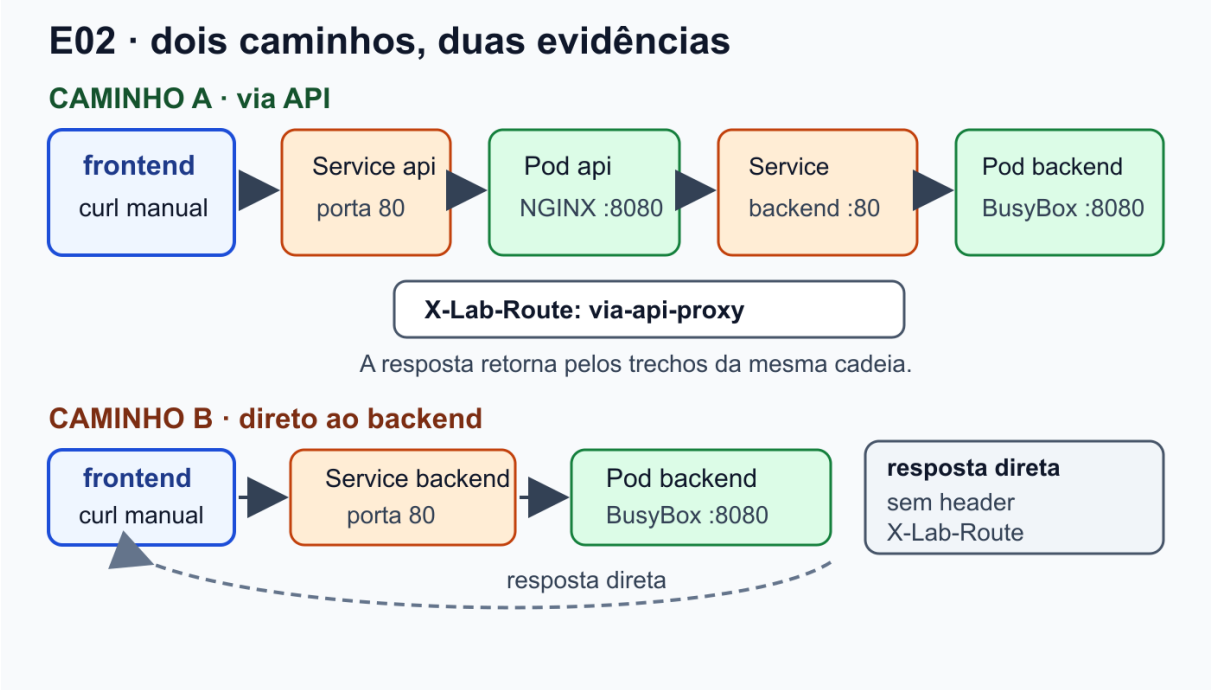
3.5 E02 — seguir uma requisição saudável

Objetivo: criar a cadeia frontend → Service api → NGINX proxy → Service backend → BusyBox httpd e comparar a resposta pela API com o acesso direto ao backend.

Pré-requisitos: E01 concluído, namespace e UID preservados, mesma revisão do guia/manifests, imagens aprovadas e disponibilidade verificada pelo instrutor. O Service backend deve existir antes de o processo NGINX da API iniciar, pois a configuração do proxy usa o nome DNS backend.

Previsão: a chamada por api/ retorna HTTP 200, conteúdo servido pelo backend e cabeçalho X-Lab-Route: via-api-proxy. Antes de E04, o acesso direto a backend/ também retorna HTTP 200, sem esse cabeçalho de proxy. O cliente chama Services na porta 80; cada Service encaminha ao Pod na porta 8080.

Figura 5 – E02 compara o caminho pelo proxy com o acesso direto



Fonte: elaboração própria (2026), com base em (Kubernetes Authors, 2026), (BusyBox Authors, 2026) e (NGINX Container Project, 2026). Nota: objetos e portas vêm dos manifests; endereços e tráfego não foram observados.

Passos explicados

1. Na janela A da VM, confira que os hashes dos YAML correspondem à revisão confirmada pelo instrutor. Hash identifica bytes daquele arquivo, não confirma que foi aplicado ou baixado no cluster:

```
RECORD_STEP='E02.1'
record 'E02 hashes dos manifests' \
  sha256sum "$LAB_DIR_REL"/*.yaml || exit 1
```

2. Crie a ServiceAccount sem permissões adicionais e as ConfigMaps. Elas guardam a página de resposta, health check e configuração do NGINX. O token automático fica desabilitado no manifesto:

```
RECORD_STEP='E02.2'
record 'E02 aplicar ConfigMaps e ServiceAccount' \
  k apply -n "$LAB_NS" -f "$LAB_DIR/00-config.yaml" \
  || exit 1
record 'E02 ServiceAccount' \
```

```
  k -n "$LAB_NS" get serviceaccount lab-workload || exit 1
record 'E02 ConfigMaps' \
  k -n "$LAB_NS" get configmaps || exit 1
```

3. Crie primeiro o Service backend e depois o Deployment. O Service fornece o nome DNS backend; o Pod BusyBox serve / e /health na porta 8080. Aguarde a probe /health:

```
RECORD_STEP='E02.3'
record 'E02 criar Service backend antes do NGINX' \
  k apply -n "$LAB_NS" \
  -f "$LAB_DIR/10-backend-service.yaml" || exit 1
record 'E02 criar Deployment backend' \
  k apply -n "$LAB_NS" \
  -f "$LAB_DIR/11-backend-deployment.yaml" || exit 1
record 'E02 aguardar probe backend' \
  k -n "$LAB_NS" rollout status deployment/backend \
  --timeout=90s || exit 1
record 'E02 Service backend' \
  k -n "$LAB_NS" get service backend || exit 1
record 'E02 EndpointSlice backend' \
  k -n "$LAB_NS" get endpointslice \
  -l kubernetes.io/service-name=backend || exit 1
```

4. Depois de confirmar o Service backend e seu estado, crie o Service api e o Deployment NGINX. A ordem garante que o nome backend existe quando NGINX carrega sua configuração. Aguarde a probe local /ready:

```
RECORD_STEP='E02.4'
record 'E02 criar Service api' \
  k apply -n "$LAB_NS" -f "$LAB_DIR/20-api-service.yaml" \
  || exit 1
record 'E02 criar Deployment api' \
  k apply -n "$LAB_NS" -f "$LAB_DIR/21-api-deployment.yaml" \
  || exit 1
record 'E02 aguardar probe local da API' \
  k -n "$LAB_NS" rollout status deployment/api \
  --timeout=90s || exit 1
```

5. Inicie o cliente ocioso frontend por último. Ele executa sleep infinity, não serve HTTP e não gera tráfego sozinho. Faça somente as duas chamadas manuais abaixo; o comando executa curl dentro do contêiner por causa do texto depois de –:

```
RECORD_STEP='E02.5'
record 'E02 criar Deployment frontend' \
  k apply -n "$LAB_NS" -f "$LAB_DIR/30-frontend-deployment.yaml" \
  || exit 1
record 'E02 aguardar frontend ocioso' \
  k -n "$LAB_NS" rollout status deployment/frontend \
  --timeout=90s || exit 1
record 'E02 UTC antes do pedido API' \
  date -u +%Y-%m-%dT%H:%M:%SZ || exit 1
if record 'E02 curl via API' \
  k exec -n "$LAB_NS" deploy/frontend -- \
  curl --fail-with-body -sS --connect-timeout 3 \
    --max-time 5 -D - http://api/; then
  rc_api=0
else
  rc_api=$?
  if (( RECORD_LOG_FAILED )); then
    printf 'Recibo falhou; pare antes do próximo pedido.\n' >&2
    exit 1
  fi
fi
printf 'E02 curl via API rc=%s\n' "$rc_api" \
  | tee -a "$LAB_RECEIPT" || exit 1
record 'E02 UTC antes do pedido direto' \
  date -u +%Y-%m-%dT%H:%M:%SZ || exit 1
if record 'E02 curl direto ao backend' \
  k exec -n "$LAB_NS" deploy/frontend -- \
  curl --fail-with-body -sS --connect-timeout 3 \
    --max-time 5 -D - http://backend/; then
  rc_direct=0
else
  rc_direct=$?
  if (( RECORD_LOG_FAILED )); then
    printf 'Recibo falhou; pare antes do próximo pedido.\n' >&2
    exit 1
  fi
```

```
fi
printf 'E02 curl direto rc=%s\n' "$rc_direct" \
  | tee -a "$LAB_RECEIPT" || exit 1
```

Leia os cabeçalhos HTTP e o corpo na saída salva no recibo. –fail-with-body mantém o corpo de erro e faz curl devolver código diferente de zero em resposta HTTP de erro. record exibe e anota o código de cada comando; rc_api e rc_direct guardam os códigos de cada pedido separadamente. curl sem –fail pode retornar zero mesmo ao receber HTTP 502.

Esperado — ILUSTRATIVO: o primeiro pedido deve exibir HTTP 200, mensagem curta do backend e X-Lab-Route: via-api-proxy; o segundo deve exibir HTTP 200 e a mesma mensagem, sem o cabeçalho X-Lab-Route. A imagem, DNS e ambiente reais podem divergir; nenhuma saída foi coletada neste guia.

Observação real — NOT_RUN: UTC: _____; revisão/hash: _____; versões do cliente/servidor: _____; Pod backend e probe: _____; Service/EndpointSlice backend: _____; Pod api e /ready: _____; cabeçalho/status/corpo por api: _____; código de saída: _____; cabeçalho/status/corpo direto: _____; código de saída: _____.

Interpretação: a diferença de cabeçalho identifica o trecho que passou pelo proxy; a resposta direta mostra que o frontend alcança o Service backend antes da policy. Um Pod Ready comprova apenas sua probe local. Uma resposta ponta a ponta reúne mais evidências do que uma lista de Pods, mas ainda não testa policy ou Hubble.

Se falhar: se a probe backend falhar, confira ConfigMap, volume, porta e eventos do Pod antes de avançar. Se backend estiver pronto mas o EndpointSlice estiver vazio, compare labels e selector. Se a API não ficar Ready, confirme que o Service backend existia antes do NGINX e examine logs da API. Se o comando curl retornar HTTP 502, registre cabeçalho, código curl e horário; não atribua causa sem evidência. Não execute curl no contêiner NGINX nem reinicie recursos protegidos.

Sucesso: os dois pedidos retornam HTTP 200 e o conteúdo do backend; apenas a resposta pela API inclui o cabeçalho do proxy; os Pods API e backend passam probes locais; Service backend precedeu a inicialização do NGINX.

Gabarito comentado: o frontend é cliente, não servidor. DNS localiza um Service, o Service seleciona destinos e NGINX abre um segundo trecho HTTP para backend. O header X-Lab-Route é adicionado pela configuração da API, logo a chamada direta não o contém. Probes não verificam a cadeia inteira.

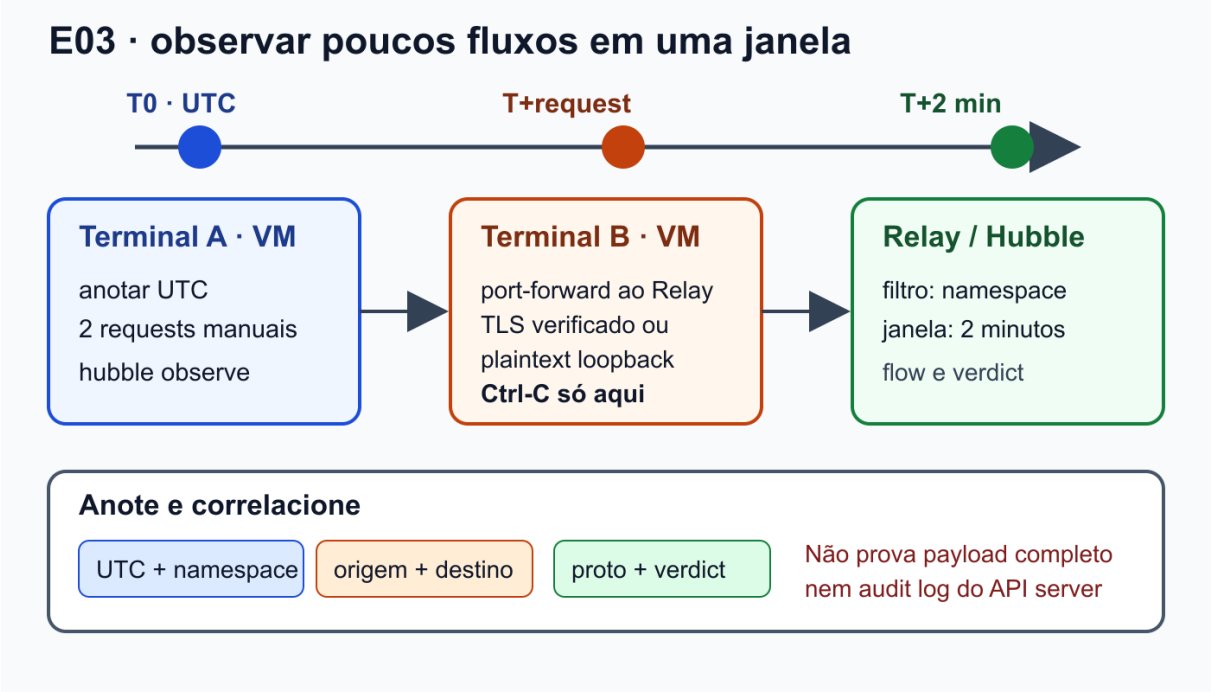
3.6 E03 — observar uma janela curta com Hubble

Objetivo: correlacionar alguns fluxos recentes com a namespace do laboratório e distinguir telemetria de fluxo de uma captura completa ou do audit log do API server.

Pré-requisitos: E02 concluído; Hubble Relay já disponível; versões CLI/Relay compatíveis e modo explicitamente declarado pelo instrutor: TLS verificado ou plaintext somente pelo port-forward configurado para loopback. Para TLS, obtenha CA, nome do servidor e eventual certificado cliente por canal seguro. Para plaintext, o instrutor confirma que o Relay o aceita e que o encaminhamento está preso a 127.0.0.1:4245. Não habilite Hubble, não instale charts, não altere o Relay e nunca ignore validação de certificado.

Previsão: uma janela de poucos minutos deve mostrar alguns fluxos do namespace, com identidades de origem/destino e verdicts quando expostos pela versão instalada. Pode haver detalhes diferentes por versão, filtro ou política. Um fluxo permitido/observado não prova o conteúdo completo de cada pacote.

Figura 6 – E03 limita a observação ao namespace e à janela do exercício



Fonte: elaboração própria (2026), com base em (Cilium Authors, 2026), incluindo setup e configuração TLS do Relay. Nota: Hubble é telemetria de fluxo; campos e retenção dependem da versão e configuração instaladas.

Passos explicados

1. Mantenha o terminal A aberto. O terminal B é uma nova sessão Bash e não herda exports do terminal A. Copie para B o caminho absoluto do kubeconfig, o contexto e a namespace Cilium fornecidos pelo instrutor; não copie o conteúdo do kubeconfig:

```
export LAB_KUBECONFIG='/caminho/absoluto/fornecido/pelo/instrutor'
export LAB_CONTEXT='contexto-fornecido-pelo-instrutor'
export LAB_CILIUM_NS='namespace-cilium-fornecido-pelo-instrutor'
if [[ ! -r "$LAB_KUBECONFIG" || -z "$LAB_CONTEXT" ||
      -z "$LAB_CILIUM_NS" ]]; then
  printf 'Falta dado de conexão validado pelo instrutor; pare.\n' >&2
  exit 1
fi
```

2. Inicie no terminal B somente o port-forward ao Relay já disponível. Confirme que o bind local usa loopback e que não escuta em interfaces externas; se não puder confirmar, pare. A conexão do cliente será configurada conforme o modo declarado. Não instale nem ative componentes:

```
cilium hubble port-forward \
  --kubeconfig "$LAB_KUBECONFIG" \
  --context "$LAB_CONTEXT" \
  -n "$LAB_CILIUM_NS"
```

3. Deixe o processo em primeiro plano no terminal B. A saída deve anunciar `127.0.0.1:4245`; confirme que não escuta em interfaces externas. Se o endereço estiver diferente ou incerto, pare. No terminal A, declare o modo que o instrutor confirmou; não tente TLS e depois plaintext automaticamente:

```
RECORD_STEP='E03.3'
export HUBBLE_SERVER='127.0.0.1:4245'
export HUBBLE_MODE='verified-tls'
```

Substitua cada valor de exemplo pelos parâmetros declarados pelo instrutor antes de executar o próximo bloco. No modo `verified-tls`, defina CA, nome do servidor e `HUBBLE_REQUIRES_CLIENT_CERT` como `yes` ou `no` conforme a exigência do Relay. Se for `yes`, defina também os caminhos locais do certificado e da chave cliente; não os copie para o recibo. No modo `loopback-plaintext`, não defina flags TLS. Essa opção só vale para a conexão do port-forward em loopback, com autorização explícita do instrutor. Um erro nunca muda o modo:

```
RECORD_STEP='E03.3'
HUBBLE_CONNECTION_ARGS=()
```

```
case "$HUBBLE_MODE" in
  verified-tls)
    export HUBBLE_TLS_CA_CERT_FILES=\
      '/caminho/absoluto/da/CA-fornecida.crt'
    export HUBBLE_TLS_SERVER_NAME=\
      'nome-do-certificado-fornecido-pelo-instrutor'
    export HUBBLE_REQUIRES_CLIENT_CERT='no'
    if [[ ! -r "$HUBBLE_TLS_CA_CERT_FILES" ||
          -z "$HUBBLE_TLS_SERVER_NAME" ]]; then
      printf 'Parâmetros TLS incompletos; pare e chame o instrutor.\n' \
        >&2
      exit 1
    fi
    HUBBLE_CONNECTION_ARGS=(
      --tls
      --tls-ca-cert-files "$HUBBLE_TLS_CA_CERT_FILES"
      --tls-server-name "$HUBBLE_TLS_SERVER_NAME"
    )
    case "$HUBBLE_REQUIRES_CLIENT_CERT" in
      no) ;;
      yes)
        if [[ ! -r "${HUBBLE_TLS_CLIENT_CERT_FILE:-}" ||
              ! -r "${HUBBLE_TLS_CLIENT_KEY_FILE:-}" ]]; then
          printf 'Certificado cliente ausente; pare.\n' \
            >&2
          exit 1
        fi
        HUBBLE_CONNECTION_ARGS+=(
          --tls-client-cert-file "$HUBBLE_TLS_CLIENT_CERT_FILE"
          --tls-client-key-file "$HUBBLE_TLS_CLIENT_KEY_FILE"
        )
        ;;
      *)
        printf 'Instrutor precisa declarar se Relay exige mTLS.\n' \
          >&2
        exit 1
        ;;
    esac
    ;;
```

```
  loopback-plaintext)
    if [[ "$HUBBLE_SERVER" != '127.0.0.1:4245' ]]; then
      printf 'Plaintext permitido somente no port-forward loopback.\n' \
        >&2
      exit 1
    fi
    ;;
  *)
    printf 'Instrutor deve declarar o modo de conexão do Relay.\n' >&2
    exit 1
    ;;
esac
record 'modo Hubble declarado pelo instrutor' \
  printf 'HUBBLE_MODE=%s\n' "$HUBBLE_MODE" || exit 1
record 'E03 status Hubble no modo declarado' \
  hubble status --server "$HUBBLE_SERVER" \
  "${HUBBLE_CONNECTION_ARGS[@]}" || exit 1
```

4. Gere poucos requests novos no terminal A, sem loop. record guarda a saída e o código de cada pedido, mesmo se um status HTTP provocar curl não zero:

```
RECORD_STEP='E03.4'
record 'E03 UTC antes do pedido API' \
  date -u +%Y-%m-%dT%H:%M:%SZ || exit 1
if record 'E03 request via API' \
  k exec -n "$LAB_NS" deploy/frontend -- \
  curl --fail-with-body -sS --connect-timeout 3 \
    --max-time 5 -D - http://api/; then
  rc_api=0
else
  rc_api=$?
  if (( RECORD_LOG_FAILED )); then
    printf 'Recibo falhou; pare antes do próximo pedido.\n' >&2
    exit 1
  fi
fi
printf 'E03 request via API rc=%s\n' "$rc_api" \
  | tee -a "$LAB_RECEIPT" || exit 1
record 'E03 UTC antes do pedido direto' \
```

```
  date -u +%Y-%m-%dT%H:%M:%SZ || exit 1
if record 'E03 request direto antes de E04' \
  k exec -n "$LAB_NS" deploy/frontend -- \
  curl --fail-with-body -sS --connect-timeout 3 \
    --max-time 5 -D - http://backend/; then
  rc_direct=0
else
  rc_direct=$?
  if (( RECORD_LOG_FAILED )); then
    printf 'Recibo falhou; pare antes do próximo pedido.\n' >&2
    exit 1
  fi
fi
printf 'E03 request direto rc=%s\n' "$rc_direct" \
  | tee -a "$LAB_RECEIPT" || exit 1
```

5. Consulte somente a namespace e uma janela curta, passando explicitamente o mesmo endpoint e os parâmetros do modo declarado usados no teste de status. Mantenha o terminal B aberto para os exercícios de policy e troubleshooting:

```
RECORD_STEP='E03.5'
record 'E03 Hubble flows no modo do E03' \
  hubble observe --server "$HUBBLE_SERVER" \
  "${HUBBLE_CONNECTION_ARGS[@]}" --namespace "$LAB_NS" \
  --since 2m || exit 1
```

6. Registre intervalo UTC, filtro, origem/destino, protocolo, verdict e campos ausentes. record salva o retorno de cada comando. Ctrl-C, somente no terminal B, encerra o encaminhamento depois de E05 ou quando o instrutor pedir.

Esperado — ILUSTRATIVO: `hubble status` conecta no modo declarado e confirma a saúde. Podem surgir fluxos recentes entre frontend e api e entre api e backend; antes da policy, também pode surgir frontend e backend direto. Hubble pode mostrar FORWARDED ou um verdict equivalente e detalhes L7 se estiverem habilitados. A saída exata, nome do processo e metadados variam; nenhuma observação real foi feita.

Observação real — NOT_RUN: versão CLI/Relay: _____; modo declarado (TLS verificado ou plaintext via loopback): _____; namespace Cilium: _____; confirmação do instrutor (sem segredo): _____; horário inicial/final UTC: _____; filtro e janela: _____;

origem/destino/protocolo/verdict: _____; pedido correlacionado: _____; código de saída: _____; campos ausentes/limitações: _____.

Interpretação: a janela ajuda a correlacionar um request recente ao fluxo de rede e à decisão observada. Fluxo permitido não prova sucesso HTTP; fluxo descartado sem endereço/tempo correlacionados não prova que foi aquele request; falta de registro pode decorrer de filtro, retenção, configuração ou intervalo.

Se falhar: erro de TLS ou conexão significa que o acesso ao Relay não foi estabelecido, não que o tráfego parou. Não altere o modo nem use fallback; pare o port-forward no terminal B com Ctrl-C e peça ao instrutor os parâmetros corretos. Se o instrutor declarou `loopback-plaintext` mas status falhou, registre o erro e pare. Se a consulta não mostrar fluxo, confirme namespace e relógio; no máximo, gere um pedido único adicional com autorização do instrutor. Se seguir ausente, preserve NOT_RUN ou inconclusivo. Não habilite observabilidade nem altere Cilium.

Sucesso: CLI conectou no modo explicitamente confirmado pelo instrutor, poucos requests foram marcados e a saída da janela filtrada foi correlacionada com tempo, namespace, origem/destino e verdict, ou a ausência de dados foi registrada como inconclusiva.

Gabarito comentado: Hubble mostra eventos de fluxo produzidos pela instrumentação e pela configuração disponíveis. O Relay não substitui captura integral nem audit log de objetos Kubernetes. Timestamp e identidade ajudam correlação; ausência de fluxo sozinho não prova ausência de tráfego.

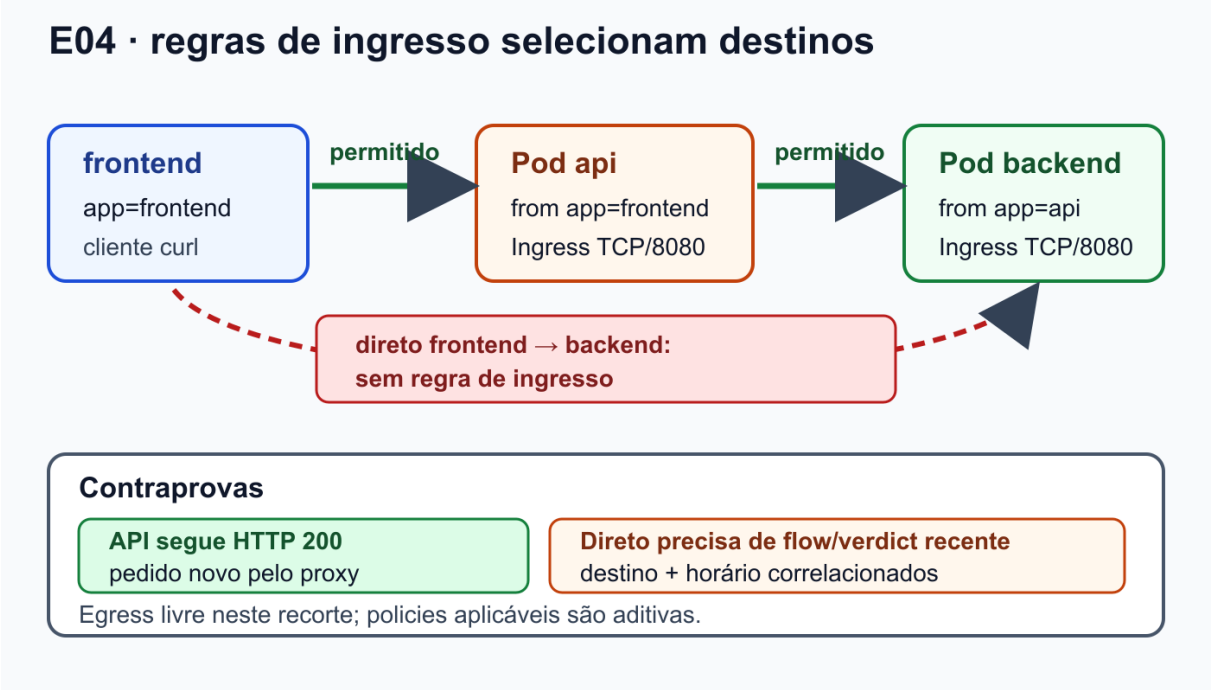
3.7 E04 — restringir ingress sem quebrar a cadeia

Objetivo: permitir somente frontend→api e api→backend por TCP/8080 na entrada dos Pods api e backend, mantendo o caminho pelo proxy e bloqueando o acesso direto do frontend ao backend.

Pré-requisitos: E02 saudável e registrada; E03 passou `hubble status` no modo de conexão declarado pelo instrutor; se plaintext foi autorizado, o server é loopback 127.0.0.1:4245; policies globais revisadas pelo instrutor; Cilium já configurado; arquivo 40-policies-ingress.yaml conferido. O exercício não altera egress, política global ou namespace.

Previsão: o pedido pela API continua saudável. O pedido direto do frontend ao backend não recebe resposta. Timeout isolado não confirma bloqueio por policy: correlacione o request novo, o destino e um verdict de descarte de policy em Hubble ou evidência equivalente.

Figura 7 – E04 permite dois trechos e bloqueia o atalho direto



Fonte: elaboração própria (2026), com base em (Kubernetes Authors, 2026), (Cilium Authors, 2026) e nos manifests E04. Nota: linha sólida/tracejada e rótulos preservam a distinção sem depender de cor.

Passos explicados

1. Confirme E02: a cadeia pela API deve estar saudável e o acesso direto ainda deve funcionar antes da policy. Se não houver baseline, resolva E02 ou pare.

2. Leia os manifests. Cada policy seleciona um destino pela label app e contém somente Ingress. O número 8080 é a porta do contêiner selecionado, não a porta 80 do Service:

```
RECORD_STEP='E04.2'
record 'E04 ler manifest de policy' \
  sed -n '1,220p' "$LAB_DIR/40-policies-ingress.yaml" \
  || exit 1
record 'E04 aplicar policies de ingresso' \
  k apply -n "$LAB_NS" -f "$LAB_DIR/40-policies-ingress.yaml" \
  || exit 1
record 'E04 policies presentes' \
  k -n "$LAB_NS" get networkpolicy || exit 1
```

3. Abra uma conexão nova para a cadeia positiva pela API. Leia status HTTP e o cabeçalho do proxy:

```
RECORD_STEP='E04.3'
record 'E04 UTC antes do pedido API' \
  date -u +%Y-%m-%dT%H:%M:%SZ || exit 1
if record 'E04 curl positivo via API' \
  k exec -n "$LAB_NS" deploy/frontend -- \
  curl --fail-with-body -sS --connect-timeout 3 \
    --max-time 5 -D - http://api/; then
  rc_api=0
else
  rc_api=$?
  if (( RECORD_LOG_FAILED )); then
    printf 'Recibo falhou; pare antes do próximo pedido.\n' >&2
    exit 1
  fi
fi
printf 'E04 curl via API rc=%s\n' "$rc_api" \
  | tee -a "$LAB_RECEIPT" || exit 1
```

4. Abra uma conexão nova para o teste direto. Um único request limitado em cinco segundos é suficiente; não faça loop:

```
RECORD_STEP='E04.4'
record 'E04 UTC antes do pedido direto' \
  date -u +%Y-%m-%dT%H:%M:%SZ || exit 1
if record 'E04 curl negativo direto ao backend' \
  k exec -n "$LAB_NS" deploy/frontend -- \
  curl --fail-with-body -sS --connect-timeout 3 \
    --max-time 5 -D - http://backend/; then
  rc_direct=0
else
  rc_direct=$?
  if (( RECORD_LOG_FAILED )); then
    printf 'Recibo falhou; pare antes do próximo pedido.\n' >&2
    exit 1
  fi
fi
printf 'E04 curl direto rc=%s\n' "$rc_direct" \
```

```
  | tee -a "$LAB_RECEIPT" || exit 1
```

5. Anote horário, saída curl, status HTTP se houve resposta e destino. Em seguida examine a janela curta de Hubble e as policies efetivamente presentes:

```
RECORD_STEP='E04.5'
record 'E04 Hubble flows no modo do E03' \
  hubble observe --server "$HUBBLE_SERVER" \
  "${HUBBLE_CONNECTION_ARGS[@]}" --namespace "$LAB_NS" \
  --since 2m || exit 1
record 'E04 objetos e labels do lab' \
  k -n "$LAB_NS" get \
  pods,services,endpointslices,networkpolicy \
  --show-labels || exit 1
```

Esperado — ILUSTRATIVO: a cadeia via API segue com HTTP 200. O acesso direto tende a não receber HTTP; o código curl pode indicar timeout ou outro erro de conexão. Correlacione a tentativa recente, o destino backend e um fluxo DROPPED / Policy denied. Policies do Kubernetes aplicáveis ao mesmo sentido são aditivas, e respostas de conexões permitidas podem retornar; outras policies ou restrições globais podem influir. (Kubernetes Authors, 2026) (Cilium Authors, 2026)

Observação real — NOT_RUN: UTC: _____; policies/namespaces: _____; resultado API (status, header, código): _____; resultado direto (status se houve, código): _____; Hubble origem/destino/verdict/tempo: _____; políticas globais ou ressalvas do instrutor: _____.

Interpretação: o Service seleciona o destino; a policy restringe quais Pods podem iniciar a conexão para aquele destino. HTTP 200 pela API verifica o caminho positivo. A falha do curl só demonstra que aquela tentativa não recebeu a resposta esperada; sem evento correlacionado, a causa é inconclusiva. Hubble não substitui o status HTTP da aplicação.

Se falhar: se o caminho via API também falhar, confira selectors, labels e ambas permissões; egress ficou fora deste recorte, mas regras globais podem afetar. Se a tentativa direta falhar sem evento correlacionado, não declare bloqueio por policy. Se ela ainda funciona, confirme que o CNI aplica policy aos Pods e que os selectors correspondem. Não remova policies amplas para testar.

Sucesso: uma chamada nova pela API permanece saudável e o acesso direto falha com evidência recente correlacionada ao backend e verdict de policy, sem mudança de egress ou política global.

Gabarito comentado: api aceita ingresso de frontend na porta 8080; backend aceita ingresso de api na 8080. O tráfego de resposta pertence a essas conexões permitidas. Frontend não é selecionado pelas duas policies, e nenhuma delas isola egress.

3.8 E05 — encontrar e restaurar um selector errado

Objetivo: introduzir uma falha controlada no selector do Service backend e mostrar por que um Pod Ready pode deixar de ser destino do Service.

Pré-requisitos: baseline E02 registrada; policies de E04 continuam aplicadas e a cadeia positiva funciona; arquivo defeituoso pertence à mesma revisão. Não faça esta falha em namespace ou cluster compartilhado.

Previsão: Pods backend podem continuar Ready enquanto o Service procura app=backend-ausente em vez da label app=backend. EndpointSlices não devem indicar destino pronto correspondente. A API pode devolver HTTP 502/504, recusa ou timeout, conforme o comportamento da rede e do proxy; o guia não prevê um status único.

Figura 8 – E05 separa a saúde do Pod da seleção pelo Service



Fonte: elaboração própria (2026), com base em (Kubernetes Authors, 2026) e no exercício E05. Nota: a árvore orienta a evidência; status HTTP isolado não identifica a causa.

Passos explicados

1. Use como baseline o pedido via API já registrado em E04; confirme que foi saudável e que o backend está Ready. Não gere outro pedido só para repetir o registro. Se a baseline já falhava, não introduza outra falha.

2. Aplique apenas o Service defeituoso. Esse arquivo muda somente o selector; não altera Pod, Deployment nem policies:

```
RECORD_STEP='E05.2'
record 'E05 aplicar selector defeituoso' \
  k apply -n "$LAB_NS" \
  -f "$LAB_DIR/41-backend-service-defeito.yaml" || exit 1
```

3. Compare labels do Pod e selector do Service. Leia também os endpoints e readiness:

```
RECORD_STEP='E05.3'
record 'E05 Pod labels' \
  k -n "$LAB_NS" get pods -l app=backend --show-labels || exit 1
record 'E05 Service selector' \
  k -n "$LAB_NS" get service backend -o yaml || exit 1
record 'E05 EndpointSlice' \
  k -n "$LAB_NS" get endpointslice \
  -l kubernetes.io/service-name=backend -o yaml || exit 1
record 'E05 readiness backend' \
  k -n "$LAB_NS" rollout status deployment/backend \
  --timeout=30s || exit 1
record 'E05 logs API recentes' \
  k -n "$LAB_NS" logs deployment/api --since=2m || exit 1
```

4. Faça uma chamada pela API, registre código de saída e status HTTP se presente. Não tente corrigir a falha usando acesso direto: E04 continua selecionando o Pod backend e bloqueando esse atalho:

```
RECORD_STEP='E05.4'
record 'E05 UTC antes do pedido com defeito' \
  date -u +%Y-%m-%dT%H:%M:%SZ || exit 1
if record 'E05 request API com selector defeituoso' \
  k exec -n "$LAB_NS" deploy/frontend -- \
  curl --fail-with-body -sS --connect-timeout 3 \
    --max-time 5 -D - http://api/; then
  rc_fault=0
else
  rc_fault=$?
```

```
  if (( RECORD_LOG_FAILED )); then
    printf 'Recibo falhou; pare antes do próximo pedido.\n' >&2
    exit 1
  fi
fi
printf 'E05 request com defeito rc=%s\n' "$rc_fault" \
  | tee -a "$LAB_RECEIPT" || exit 1
```

5. Restaure somente o Service saudável e repita os dois controles. Não reaplique tudo e não reinicie Deployment:

```
RECORD_STEP='E05.5'
record 'E05 restaurar Service saudável' \
  k apply -n "$LAB_NS" \
  -f "$LAB_DIR/10-backend-service.yaml" || exit 1
record 'E05 verificar Service restaurado' \
  k -n "$LAB_NS" get service backend -o yaml || exit 1
record 'E05 verificar EndpointSlice restaurado' \
  k -n "$LAB_NS" get endpointslice \
  -l kubernetes.io/service-name=backend || exit 1
record 'E05 UTC antes do pedido API restaurado' \
  date -u +%Y-%m-%dT%H:%M:%SZ || exit 1
if record 'E05 request API após restauração' \
  k exec -n "$LAB_NS" deploy/frontend -- \
  curl --fail-with-body -sS --connect-timeout 3 \
    --max-time 5 -D - http://api/; then
  rc_api=0
else
  rc_api=$?
  if (( RECORD_LOG_FAILED )); then
    printf 'Recibo falhou; pare antes do próximo pedido.\n' >&2
    exit 1
  fi
fi
printf 'E05 request API restaurado rc=%s\n' "$rc_api" \
  | tee -a "$LAB_RECEIPT" || exit 1
record 'E05 UTC antes do pedido direto' \
  date -u +%Y-%m-%dT%H:%M:%SZ || exit 1
if record 'E05 request direto sob policy' \
```

```
  k exec -n "$LAB_NS" deploy/frontend -- \
  curl --fail-with-body -sS --connect-timeout 3 \
    --max-time 5 -D - http://backend/; then
  rc_direct=0
else
  rc_direct=$?
  if (( RECORD_LOG_FAILED )); then
    printf 'Recibo falhou; pare antes do próximo pedido.\n' >&2
    exit 1
  fi
fi
printf 'E05 request direto rc=%s\n' "$rc_direct" \
  | tee -a "$LAB_RECEIPT" || exit 1
```

Esperado — ILUSTRATIVO: o Pod backend permanece Ready; sem correspondência de label, o Service perde destino correspondente e a chamada via API pode falhar. Depois da restauração, o EndpointSlice volta a refletir o Pod pronto, a chamada via API retorna HTTP 200 e a chamada direta continua bloqueada pela policy. EndpointSlices são gerados a partir dos Pods que correspondem ao selector do Service. (Kubernetes Authors, 2026)

Observação real — NOT_RUN: baseline UTC: _____; labels do Pod: _____; selector antes/depois: _____; destino(s)/condição no EndpointSlice: _____; readiness: _____; status HTTP e código de saída durante a falha: _____; Service restaurado: _____; status/header via API: _____; status/código direto pós-policy: _____.

Interpretação: readiness é uma pergunta local sobre a probe do Pod; selector e EndpointSlice descrevem a seleção feita pelo Service. Um processo saudável pode estar sem destino associado. A restauração de um único objeto testa se a hipótese explica a falha sem remover as policies.

Se falhar: se a probe backend falhar, pare: não é apenas uma falha de selector. Se EndpointSlice ainda tem destino, compare label, selector, Service e condição do endpoint. Se a chamada não falhar, preserve essa observação e peça ajuda ao instrutor; pode existir outro Service ou resolução diferente. Se restaurar não devolver destino, confira somente o YAML saudável e Pod Ready; não execute rollout restart, delete all ou force.

Sucesso: a sequência Pod Ready → selector não corresponde → nenhum destino pronto → chamada via API falha é observável; restaurar somente o Service recupera o caminho permitido e mantém o acesso direto negado.

Gabarito comentado: um Deployment mantém o Pod, mas o Service escolhe por labels. EndpointSlice expõe destinos que o control plane associou ao Service. O selector errado inter-

rompe a seleção lógica sem exigir editar ou matar o Pod. A resposta HTTP específica depende da implementação e não prova sozinha a causa.

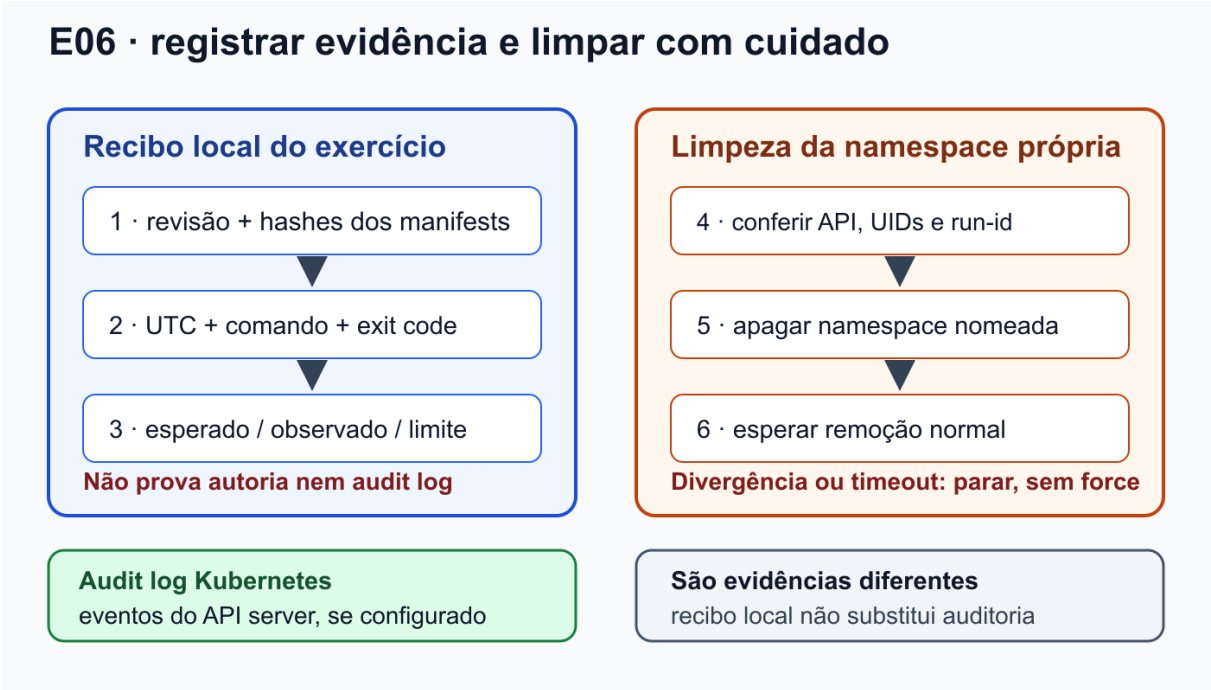
3.9 E06 — auditar evidências e limpar somente o próprio namespace

Objetivo: produzir um recibo local mínimo que distingue intenção, observação e conclusão, e remover somente a namespace desta rodada após confirmar UID e run-id.

Pré-requisitos: registros E01–E05; terminal A mantém o mesmo contexto, namespace, run-id e UID; todas as políticas e objetos do laboratório foram inventariados; instrutor está disponível. Se a sessão do shell foi perdida ou a identidade não confere, não execute cleanup.

Previsão: o recibo local identifica revisão, hashes, horário UTC, versões, identidade, etapas e códigos, resultado esperado e observado, limites e cleanup. Não contém kubeconfig, token, certificado, chave, Secret ou dados pessoais. A deleção da namespace remove os recursos namespaced criados nela; não remove objetos globais.

Figura 9 – E06 mantém o recibo local separado do audit log do control plane



Fonte: elaboração própria (2026), com base em (Kubernetes Authors, 2026) e no processo de evidência E06. Nota: o exercício não habilita, altera ou coleta audit log do control plane.

Passos explicados

1. E01 criou o arquivo local `receipt.md` antes da primeira chamada ao API server, com permissões restritas. Confira que o mesmo arquivo está presente; não crie um segundo

recibo. Os comandos `record` seguintes acrescentam revisão do guia, exercício e passo, rótulo, saída e código de saída. Não escrevem os argumentos expandidos, que podem conter caminhos locais. O modo 700 restringe a leitura no sistema de arquivos, mas não prova autoria nem fornece custódia imutável:

```
RECORD_STEP='E06.1'
if [[ ! -f "$LAB_RECEIPT" || ! -d "$LAB_RECEIPT_DIR" ]]; then
  printf 'Recibo E01 ausente; pare e chame o instrutor.\n' >&2
  exit 1
fi
record 'E06 hora UTC' date -u +%Y-%m-%dT%H:%M:%SZ || exit 1
record 'E06 versão Kubernetes' k version || exit 1
record 'E06 hashes do guia e manifests' \
  sha256sum "$LAB_GUIDE_REL" "$LAB_DIR_REL"/*.yaml || exit 1
record 'E06 inventário final com labels' \
  k -n "$LAB_NS" get \
  pods,services,endpointslices,networkpolicy --show-labels || exit 1
pod_columns='custom-columns=NAME:.metadata.name,'
pod_columns+='IMAGE:.spec.containers[*].image,'
pod_columns+='IMAGE_ID:.status.containerStatuses[*].imageID,'
pod_columns+='NODE:.spec.nodeName'
node_columns='custom-columns=NAME:.metadata.name,'
node_columns+='ARCH:.status.nodeInfo.architecture,'
node_columns+='OS:.status.nodeInfo.osImage'
if ! record 'E06 imageIDs e nós dos Pods' \
  k -n "$LAB_NS" get pods -o "$pod_columns"; then
  if (( RECORD_LOG_FAILED )); then
    printf 'Recibo falhou; pare antes de continuar.\n' >&2
    exit 1
  fi
  printf 'ImageIDs não coletados; marque NOT_RUN e explique.\n' \
    | tee -a "$LAB_RECEIPT" || exit 1
fi
if ! record 'E06 arquitetura e SO dos nós' \
  k get nodes -o "$node_columns"; then
  if (( RECORD_LOG_FAILED )); then
    printf 'Recibo falhou; pare antes de continuar.\n' >&2
    exit 1
  fi
  printf 'Arquitetura não coletada; marque NOT_RUN e explique.\n' \
```

```
    | tee -a "$LAB_RECEIPT" || exit 1
fi
```

2. E01 já registrou `kubectl version`; o status Hubble do E03 registra a versão anunciada pelo Relay. Confirme esses registros e acrescente a revisão, versão do guia e conclusão. Para cada observação, mantenha rótulo da etapa, UTC, código de saída, status HTTP se houver, valor esperado, saída observada e conclusão limitada. Registre NOT_RUN ou inconclusivo quando uma verificação não ocorreu. Não copie kubeconfig, token, certificados, chaves, Secrets ou caminhos pessoais para uma cópia compartilhada do recibo.

3. Antes de apagar, compare mais uma vez o API server e UID kube-system com o recibo, depois compare o UID e a label run-id atuais com os valores da sessão. Os testes encerram o shell com erro em qualquer divergência:

```
actual_context="$(k config view --minify \
  -o jsonpath='{.contexts[0].name}')" || exit 1
actual_server="$(k config view --minify \
  -o jsonpath='{.clusters[0].cluster.server}')" || exit 1
actual_cluster_uid="$(k get namespace kube-system \
  -o jsonpath='{.metadata.uid}')" || exit 1
actual_ns_uid="$(k get namespace "$LAB_NS" \
  -o jsonpath='{.metadata.uid}')" || exit 1
actual_run_id="$(k get namespace "$LAB_NS" \
  -o jsonpath='{.metadata.labels.lab\.learn-cilium\.org/run-id}')" \
  || exit 1
if [[ "$actual_context" != "$LAB_CONTEXT" ||
      "$actual_server" != "$LAB_API_SERVER" ||
      "$actual_cluster_uid" != "$LAB_CLUSTER_UID" ||
      "$actual_ns_uid" != "$LAB_NS_UID" ||
      "$actual_run_id" != "$LAB_RUN_ID" ]]; then
  printf 'Identidade divergente; não apague. Chame o instrutor.\n' >&2
  exit 1
fi
```

4. Se todos os valores conferirem e o instrutor liberar o encerramento, peça ao Kubernetes para apagar somente a namespace nominal. Não use –force, finalizers, delete all, selector amplo ou política global:

```
RECORD_STEP='E06.4'
record 'E06 UTC antes do cleanup' \
  date -u +%Y-%m-%dT%H:%M:%SZ || exit 1
record 'E06 solicitar delete sem espera implícita' \
  k delete namespace "$LAB_NS" --wait=false || exit 1
record 'E06 aguardar remoção limitada a 120s' \
  k wait --for=delete "namespace/$LAB_NS" \
  --timeout=120s || exit 1
remaining_namespaces="$(k get namespaces -o name)" || exit 1
printf 'E06 conferência final de namespace exit_code=0\n' \
  | tee -a "$LAB_RECEIPT" || exit 1
if printf '%s\n' "$remaining_namespaces" \
  | grep -Fxq -- "namespace/$LAB_NS"; then
  printf 'Namespace ainda aparece; não force remoção.\n' >&2
  exit 1
fi
printf 'Namespace própria ausente após o wait.\n' \
  | tee -a "$LAB_RECEIPT" || exit 1
```

5. Anote hora e código de saída. Se houver timeout, pare e comunique o instrutor; não force finalizers. O wait confirma a remoção observável do objeto namespace, não auditoria imutável nem exclusão de cópias do recibo local.

Esperado — ILUSTRATIVO: o recibo contém a trilha pretendida e observada sem credenciais; os guardas comparam contexto, endpoint, UIDs e run-id; a namespace própria desaparece após o pedido de remoção. Nada neste exercício prova que o API server tinha audit habilitado.

Observação real — NOT_RUN: diretório/controle de acesso do recibo: _____; revisão e hashes: _____; versões e identidade: _____; comandos/códigos de saída: _____; esperado vs observado: _____; conclusão/limitações: _____; verificações UID/run-id antes do cleanup: _____; saída e UTC do wait: _____; namespace ausente: _____.

Interpretação: o recibo local é uma nota produzida pelo participante; ele ajuda outra pessoa a seguir o raciocínio se seus limites e lacunas forem explícitos. Audit log Kubernetes é registro do API server e depende de configuração e política do cluster. Uma coisa não substitui a outra. UID e run-id tornam a limpeza mais estreita, mas não autorizam remover qualquer recurso fora da namespace própria.

Se falhar: se a identidade, UID ou label divergir, não apague e chame o instrutor. Se o

wait exceder o tempo, não force nem remova finalizers. Se não conseguir confirmar ausência, registre resultado inconclusivo. Se o recibo contiver credencial, não o distribua; peça orientação para tratamento seguro.

Sucesso: recibo local separa esperado de observado, contém hashes/horas/versões/etapas/códigos e limitações sem segredo; a namespace só foi apagada após reconferir contexto, cluster, UID e run-id; o wait e a lista final confirmaram ausência.

Gabarito comentado: log local registra o que o participante escolheu anotar; audit log do API server registra eventos conforme a configuração administrativa. O comando de limpeza destrói apenas a namespace confirmada e espera sua remoção normal. Sem uma confirmação de auditoria do control plane, não se pode afirmar que eventos de API foram retidos.

Perguntas para recordar

1. Qual a diferença entre readiness de um Pod e a existência de um destino pronto no EndpointSlice?
2. Por que o Service backend deve preceder a inicialização do NGINX da API neste cenário?
3. O que um erro curl prova, e que evidência adicional ajuda a atribuí-lo à policy?
4. Um recibo local de comandos é um audit log do API server?

Gabarito comentado: (1) readiness é a resposta à probe configurada no Pod; o EndpointSlice contém destinos associados ao selector do Service. (2) a configuração NGINX consulta o nome DNS backend no início; o Service cria esse nome estável antes do Pod proxy iniciar. (3) prova que aquela tentativa não obteve a resposta esperada; timestamp, destino e verdict correlacionados ajudam a entender a falha. (4) não; o audit log é produzido pelo API server segundo configuração administrativa, o recibo é um documento local.

Erro comum: chamar todo erro de rede de "policy drop" ou limpar recursos sem confirmar identidade. Colete a próxima evidência que separa hipóteses; não aumente o alcance de uma correção para tornar o sintoma desaparecer.

Resumo: o caminho tem várias camadas: processo, DNS, Service, EndpointSlice, proxy, Pod e policy. Cada ferramenta responde uma pergunta limitada. Uma chamada positiva, uma tentativa negativa correlacionada, a restauração estreita e uma limpeza guardada completam a trilha.

Glossário: Deployment — mantém Pods declarados; Service — nome e seleção lógica de destinos; EndpointSlice — lista parcial de destinos do Service; label — chave/valor usado para agrupar; selector — expressão que escolhe objetos; ConfigMap — configuração não secreta; probe — verificação local de aplicação; NetworkPolicy — permissão de tráfego para Pods selecionados; Hubble — observabilidade de fluxos Cilium; audit log — eventos de API server conforme política do cluster.

Leituras: probes, Services, DNS, EndpointSlices, NetworkPolicy, segurança de Pods e auditoria no Kubernetes; Hubble e troubleshooting no Cilium.

PRÉVIA: COMO EBPF RESPONDE A EVENTOS

Esta prévia acompanha um programa eBPF — um programa restrito que o kernel Linux pode carregar — do pedido de carga até um evento. O kernel é o núcleo do Linux; hook é um ponto previsto de ligação, e evento é a ocorrência que chega àquele contexto. Espaço de usuário é a área fora do kernel onde processos executam; ali, o agente do Cilium configura componentes, enquanto o programa ligado ao hook atua no contexto do kernel. O caminho exato depende do tipo de programa e da configuração.

Vocabulário de configuração: YAML é um formato textual para dados. No Kubernetes, label (rótulo) é um par chave/valor de metadados que ajuda a identificar e selecionar objetos, mas não autentica; no exemplo ilustrativo `app: api`, `app` é a chave e `api` é o valor (Kubernetes Authors, 2026). Um manifesto Kubernetes é um documento declarativo que descreve um recurso; policy (política de tráfego) é uma regra declarada para controlar comunicações (Cilium Authors, 2026). Nas citações autor-data, 2026 é o ano de consulta quando a página não informa o ano de publicação; `[s.d.]` nas referências conserva essa ausência sem inventar data.

As figuras são modelos didáticos originais. O fragmento YAML e os nomes `frontend`, `api` e `backend` são exemplos ilustrativos; não descrevem um cluster observado. A documentação stable do Cilium consultada em 28 de outubro de 2026 indicava 1.20.4; isso é referência documental, não versão instalada ou executada.

Um programa eBPF encontra seu evento

O que você vai aprender

Ao final, você poderá:

- explicar com palavras simples o que é um hook e onde um programa eBPF se conecta;
- ordenar solicitação de carga, verificação, aceitação ou rejeição, attach e execução por evento;
- distinguir o que a aprovação do verifier indica daquilo que ela não prova, e separar o agente Cilium do datapath.

Pré-requisito: nenhum cluster ou terminal. Leia a sequência como um modelo conceitual; ela não é uma receita de instalação.

Palavras antes do jargão

Quadro 1 – Vocabulário em uma frase

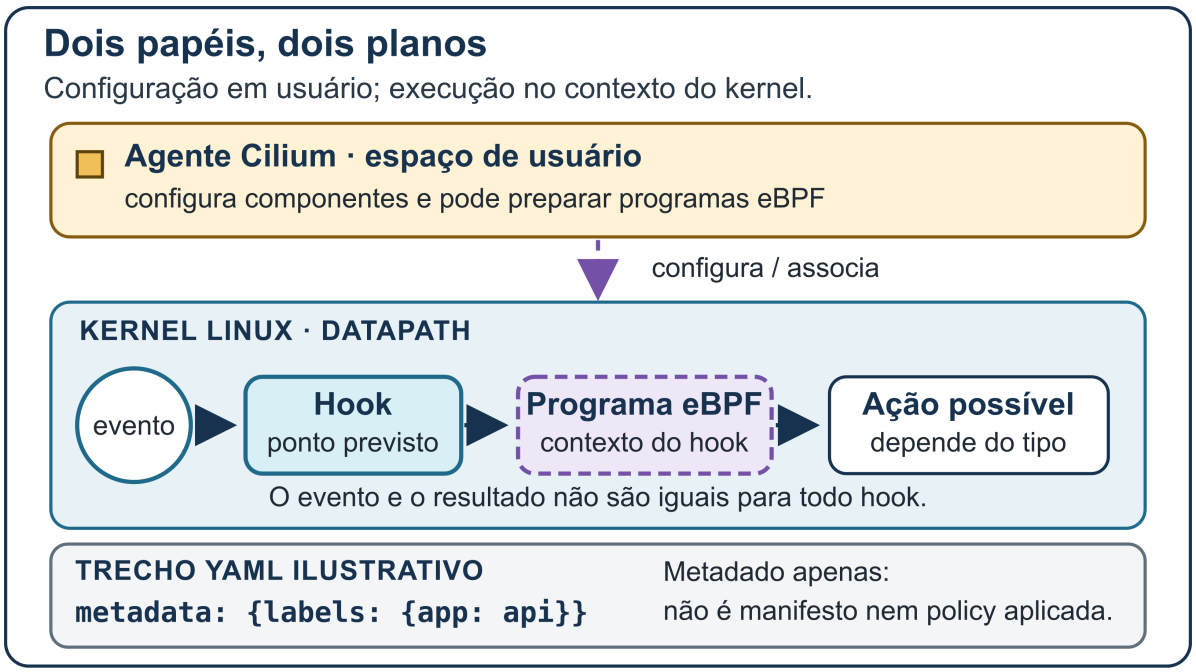
Termo	**Definição curta e limite**
Kernel / eBPF	O kernel é o núcleo do Linux; eBPF carrega programas restritos que respondem a certos eventos.
Hook / evento	Hook é um ponto previsto de ligação; evento é a ocorrência que chega a esse contexto.
Verifier / mapa	Verifier analisa propriedades de segurança na carga; mapa guarda valores associados a chaves.
Attach / JIT	Attach liga um programa aceito ao hook; JIT compila para instruções nativas quando aplicável.
Agente / datapath	O agente Cilium configura componentes; o datapath processa tráfego. Cada um tem um papel diferente.

Fonte: Elaboração própria (2026), com base em (eBPF Foundation, 2026), (Linux Kernel Documentation, 2026) e (Cilium Authors, 2026).

Um ponto de ligação, dois papéis

A analogia do hook como ponto de conexão ajuda a localizar o attach; ele não é um guarda geral, e contexto e resultado variam por tipo.

Figura 10 – O agente configura; o programa atua no hook



Fonte: Elaboração própria (2026), com base em (Cilium Authors, 2026), (eBPF Foundation, 2026) e (Kubernetes Authors, 2026).

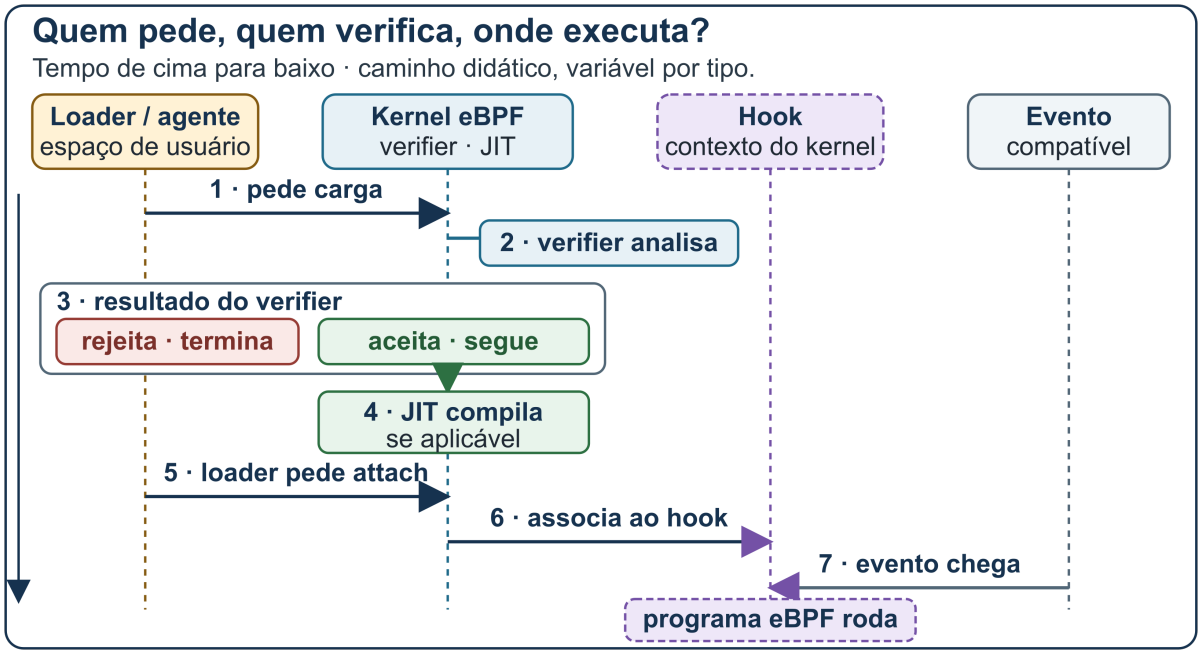
Leitura da Figura 10: agente configura; no kernel, evento → hook → programa → ação por tipo. O fragmento YAML `metadata: {labels: {app: api}}` é ilustrativo, não manifesto nem policy aplicada; label não autentica. (Kubernetes Authors, 2026).

Da solicitação ao evento

No modelo temporal, um loader em espaço de usuário solicita a carga ao kernel. O verifier do kernel analisa o programa; se rejeita, a tentativa termina. Se aceita, o kernel pode compilar com JIT quando aplicável. Então o loader solicita attach, o kernel associa o programa ao hook e um evento compatível chega ao hook, onde o programa executa no contexto do kernel. Esta é uma sequência didática, pois os mecanismos de carga e attach variam por tipo de programa. (eBPF Foundation, 2026; Linux Kernel Documentation, 2026; Cilium Authors, 2026)

No caminho rejeitado, a tentativa termina; no caminho aceito, JIT permanece condicional e o loader pede attach ao kernel.

Figura 11 – Sequência temporal: loader, kernel e hook



Fonte: Elaboração própria (2026), com base em (eBPF Foundation, 2026) e (Linux Kernel Documentation, 2026).

Aceito ou rejeitado: o alcance do verifier

O verifier analisa tipos, estado, acessos à memória e caminhos. Se não prova as condições aplicáveis, rejeita a carga. Se aceita, o programa passou nessas verificações; isso não confirma a intenção lógica. (Linux Kernel Documentation, 2026).

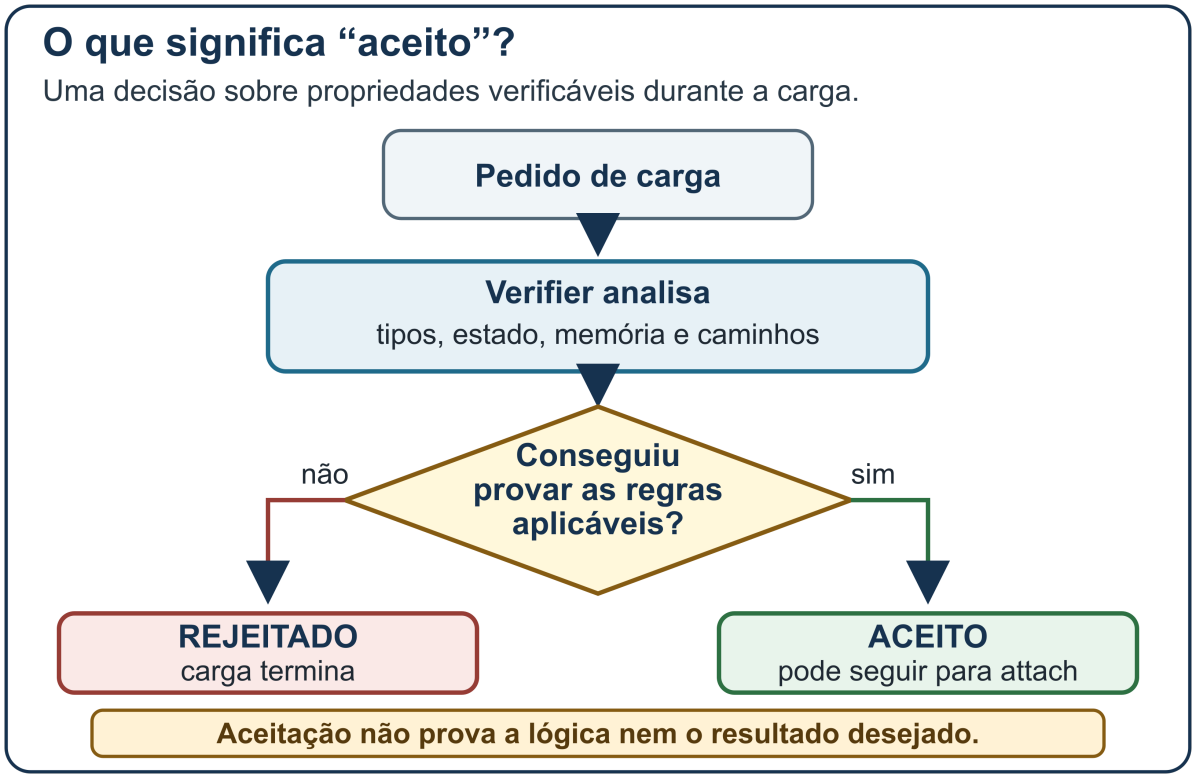
Leitura do fluxo: se as propriedades não são provadas, a carga é rejeitada; se são, o programa pode seguir para attach, sem validar a lógica desejada.

Tabela 1 – Verificações e limites da conclusão

O verifier analisa	Isso ajuda a estabelecer	Continua sem prova
Tipos e estado	uso compatível dos valores acompanhados	que a regra de negócio é a desejada
Acessos à memória	respeito aos limites verificados	que um serviço respondeu corretamente
Caminhos do programa	que os caminhos analisados atendem às restrições	que o tráfego terá o resultado esperado

Fonte: Elaboração própria (2026), com base em (Linux Kernel Documentation, 2026).

Figura 12 – Decisão do verifier e limite da conclusão



Fonte: Elaboração própria (2026), com base em (Linux Kernel Documentation, 2026).

Pratique e confira

Exercício conceitual (15 minutos). Ordene: (B) solicitar carga, (D) verifier, (F) aceitar/rejeitar, (A) attach, (C) evento e (E) execução. Identifique qual etapa aparece como condicional e em que ponto da sequência ocorre; não execute comandos.

Gabarito comentado: B → D → F; rejeitado, termina. Aceito: a etapa JIT aparece depois da aceitação e antes do attach, quando aplicável; então vêm attach, evento e execução (A → C → E). Isso não prova que a lógica é correta.

Perguntas para recordar e transferir: (1) Hook e evento são iguais? (2) O que o verifier analisa? (3) O que a aceitação não prova? (4) Como diferem agente e datapath?

Erro comum: “O verifier aceitou, então a policy está correta.” A conclusão não decorre da verificação: uma análise de segurança de execução não valida, por si só, a intenção lógica ou o comportamento completo da aplicação.

Resumo: o agente configura componentes; o kernel verifica a carga; attach conecta programa e hook; um evento compatível pode executar o programa; aceitar não prova a lógica.

REFERÊNCIAS

BusyBox Authors. HTTP server applet (`httpd`) no manual BusyBox. [s.d.]. Acesso em: 28 out. 2026. Disponível em: BusyBox manual.

Cilium Authors. Documentação stable 1.20.4: troubleshooting, Hubble Relay, TLS, Network Policy e observabilidade. [s.d.]. Acesso em: 28 out. 2026. Disponível em: troubleshooting, Hubble setup, Hubble TLS, network policy.

eBPF Foundation. What is eBPF? [s.d.]. Acesso em: 28 out. 2026. Disponível em: ebpf.io/what-is-ebpf.

GNU Project. Bash Reference Manual, versão 5.3. 2025. Disponível em: quoting, shell parameters, exit status. Acesso em: 28 out. 2026.

IETF. RFC 1034: Domain Names — Concepts and Facilities. 1987. Disponível em: RFC 1034. Acesso em: 28 out. 2026.

IETF. RFC 4632: Classless Inter-domain Routing (CIDR). 2006. Disponível em: RFC 4632. Acesso em: 28 out. 2026.

IETF. RFC 9110: HTTP Semantics; RFC 9293: Transmission Control Protocol (TCP). 2022. Disponível em: RFC 9110 e RFC 9293. Acesso em: 28 out. 2026.

Kubernetes Authors. Documentação: labels/selectors, Pods, Services, DNS, EndpointSlices, probes, NetworkPolicy, segurança de Pods e audit logging. [s.d.]. Acesso em: 28 out. 2026. Disponível em: labels, Pods, Services, DNS, EndpointSlices, probes, NetworkPolicy, security context e audit logging.

Linux Kernel Documentation. BPF verifier e BPF maps. [s.d.]. Acesso em: 28 out. 2026. Disponível em: BPF verifier e BPF maps.

Linux man-pages project. namespaces(7) e cgroups(7). [s.d.]. Acesso em: 28 out. 2026. Disponível em: namespaces e cgroups.

NGINX Container Project. NGINX unprivileged container: uso e diretórios graváveis. [s.d.]. Acesso em: 28 out. 2026. Disponível em: README.